

Kafka + Event-Driven Systems — Interview Prep Guide

Grounded in `Mike.Dev.Kafka.sln` — for a 2–4 year experience level

This document covers two kinds of material: - **Implemented concepts** — things this solution actually does, with real file references, so you can talk about them as "things I built," not "things I read about." - **Conceptual (not implemented)** — things worth understanding at this experience level even though this codebase doesn't exercise them. These are marked clearly so you don't misrepresent them in an interview.

Every major section ends with a **Q&A block** styled like real interview questions. Read the theory, then try answering the questions cold before checking the model answer.

Table of Contents

1. [Solution Architecture Recap](#)
2. [Core Kafka Concepts](#)
3. [Why Kafka Is Fast](#)
4. [Kafka vs. Traditional Message Queues](#)
5. [Producers](#)
6. [Consumers](#)
7. [Message Headers, Correlation IDs & Tracing](#)
8. [The Outbox Pattern](#)
9. [Exactly-Once Semantics & Transactions](#)
10. [Kafka Transactions vs. Two-Phase Commit](#)
11. [Retry Strategy](#)
12. [Dead Letter Topics](#)
13. [Idempotent Consumers](#)
14. [Large Messages & the Claim-Check Pattern](#)
15. [Schema Registry & Schema Evolution](#)
16. [Delivery Semantics — The Full Picture](#)

17. [Partitioning & Ordering \(Conceptual\)](#)
 18. [Consumer Group Rebalancing \(Conceptual\)](#)
 19. [Cluster, Replication & Broker Internals \(Conceptual\)](#)
 20. [Kafka Streams & ksqlDB \(Conceptual\)](#)
 21. [Kafka Connect & CDC \(Conceptual\)](#)
 22. [Security \(Conceptual\)](#)
 23. [Monitoring & Operations](#)
 24. [Testing Kafka Applications](#)
 25. [Docker/KRaft Setup — What We Actually Hit](#)
 26. [Real Bugs We Debugged — Interview Gold](#)
 27. [Rapid-Fire Q&A Catalog](#)
-

1. Solution Architecture Recap

Four projects:

- `Mike.Dev.Kafka.Contracts` — shared `DeviceEvent` record (the message contract).
- `Mike.Dev.Kafka.BuildingBlocks` — reusable Kafka infrastructure: producer, transactional producer, consumer, retry, dead-letter, serialization, schema registry integration.
- `Mike.Dev.Kafka.Producer` — writes `DeviceEvent`s via the **Outbox pattern**: a Postgres table + EF Core, drained by a background dispatcher that publishes to Kafka.
- `Mike.Dev.Kafka.Consumer` — consumes `device-events`, processes with retry + idempotency, and republishes an audit event using **exactly-once semantics** (EOS), with dead-lettering on failure.

Flow:

```
[App writes DeviceEvent] → [Postgres outbox table, same DB txn as business write]
    ↓ (KafkaOutboxDispatcher, background poll)
[device-events topic] ← Schema Registry validates/registers schema
    ↓ (DeviceEventConsumer)
[Idempotency check] → [Business handling] → [Transactional produce to device-events.audit
    + consumer offset commit, same Kafka txn]
    ↓ (on any unrecoverable failure)
[device-events.DLT topic]
```

Three Kafka topics: `device-events` (primary), `device-events.audit` (EOS output), `device-events.DLT` (dead letters). Each has its own registered JSON Schema subject in Schema Registry: `device-events-`

`value` , `device-events.audit-value` , `device-events.DLT-value` .

2. Core Kafka Concepts

Theory

- **Broker:** a single Kafka server. Holds partitions, serves produce/consume requests.
- **Cluster:** a set of brokers working together, coordinated via KRaft (modern) or ZooKeeper (legacy, deprecated as of Kafka 4.0).
- **Topic:** a named, append-only log. Logical grouping of messages (e.g. `device-events`).
- **Partition:** a topic is split into 1+ partitions. Each partition is an ordered, immutable sequence of messages, each with a monotonically increasing **offset**. Partitions are the unit of parallelism — one partition can only be actively read by one consumer within a given consumer group at a time.
- **Offset:** a partition-local, monotonically increasing integer identifying a message's position. Not global across partitions.
- **Consumer Group:** a named set of consumers cooperating to consume a topic; Kafka assigns each partition to exactly one consumer within the group. This is how Kafka achieves both parallelism (multiple consumers) and safety (no two consumers in the same group double-process a partition).
- **Replication:** each partition has a **replication factor** (N copies across brokers). One replica is the **leader** (handles all reads/writes); others are **followers** that replicate the leader's log.
- **ISR (In-Sync Replicas):** the subset of replicas that are fully caught up with the leader. Only ISR members are eligible to become the new leader if the current leader fails, without data loss.
- **Retention:** how long a topic keeps messages — time-based (`retention.ms`) or size-based (`retention.bytes`). Independent of whether messages were consumed; Kafka doesn't delete on-read like a traditional queue.
- **Log Compaction:** an alternative cleanup policy (`cleanup.policy=compact`) that keeps only the latest message per key, forever, instead of deleting by age. Used for topics that represent "current state" (e.g. `__consumer_offsets` , KTables).

Where this shows up in our solution

- `Mike.Dev.Kafka.Consumer/appsettings.json` sets `IsolationLevel: ReadCommitted` — this is a *consumer-side* setting that only matters when *producers* use transactions (see §9). It hides messages from aborted/in-flight transactions.
- Our topics run with Kafka's defaults: 1 partition, replication factor 1 (single-broker dev setup — see §19 for why that matters in production).

Q&A

Q: What's the difference between a topic and a partition? A: A topic is the logical name/category consumers and producers agree on. A partition is a physical, ordered log — the topic is split across 1+ partitions to allow parallel reads/writes. Ordering is only guaranteed *within* a partition, not across the whole topic.

Q: Why can't you have more active consumers in a group than partitions? A: Kafka assigns each partition to exactly one consumer within a group, to avoid double-processing. If you have more consumers than partitions, the extras sit idle. This is why partition count is effectively your parallelism ceiling for a consumer group.

Q: What happens to a message once it's consumed — is it deleted? A: No. Kafka doesn't delete on read. Messages persist until retention (time or size based) expires them, or forever under log compaction (keeping only the latest per key). Multiple consumer groups can independently re-read the same messages at their own pace, each tracking their own offsets.

Q: What is an offset, and is it global? A: A per-partition, monotonically increasing integer marking a message's position in that partition's log. It's *not* global across partitions — offset 5 in partition 0 and offset 5 in partition 1 are unrelated messages.

Q: What's the difference between retention and log compaction? A: Retention deletes by age/size regardless of key. Compaction keeps only the most recent message per key indefinitely, deleting older messages with the same key. Compaction is for "latest state" topics; retention is for "event stream, keep N days" topics.

Q: What is `__consumer_offsets` and why is it a compacted topic? A: An internal Kafka topic where consumer group offset commits are themselves stored (as Kafka messages — `(group, topic, partition) → offset`). It's compacted because only the *latest* committed offset per group/topic/partition matters — history of previous offsets is useless, so compaction (keep latest per key, drop the rest) is exactly the right cleanup policy, unlike a normal event-stream topic where you want to retain history.

Q: Can a topic have zero replication (replication factor 1) safely? A: It "works" (this project runs that way for local dev), but it means a single broker failure loses that partition's data entirely — there's no follower copy to fail over to. Never acceptable for production data you care about; RF=3 is the common baseline.

3. Why Kafka Is Fast

Conceptual — this project never needed to reason about Kafka's internal I/O performance, but "why is Kafka fast" is a very common interview question and worth understanding at a mechanical level, not just "it's designed for high throughput."

Theory

- **Sequential disk I/O:** Kafka's log is append-only — every write goes to the end of the active segment file. Sequential writes (and reads, when consumers are caught up and reading the tail) are dramatically cheaper than random I/O, especially avoiding seek overhead. This is the foundational design choice that makes a "distributed log" a viable high-throughput primitive instead of needing complex in-place update data structures like a B-tree.
- **OS page cache reliance:** Kafka deliberately does *not* maintain its own large in-process cache of message data. It relies on the operating system's page cache — data written to a log segment is written through the OS page cache (and flushed to disk asynchronously by the OS), and reads for data still in the page cache are served without touching disk at all. This avoids **double buffering** (the same bytes cached twice — once by Kafka's own process, once by the OS) and avoids GC pressure in the JVM from managing a huge in-process cache.
- **Zero-copy transfer (`sendfile()`):** When a consumer's fetch request can be served from an already-page-cached log segment, Kafka uses the `sendfile()` system call to transfer bytes directly from the file (page cache) to the network socket, entirely in kernel space — without ever copying the data into the Kafka process's own (JVM heap) memory. A naive implementation would copy disk→app memory→socket buffer (multiple copies, context switches); zero-copy skips the middle hop entirely.
- **Batching & compression** (see §5): amortizes per-request network/processing overhead across many messages, and compressing a larger batch achieves a better ratio than compressing messages individually.
- **Partitioning:** horizontal parallelism — more partitions spread across more brokers means more total I/O throughput and more possible concurrent consumers.

Q&A

Q: Why does Kafka rely on the OS page cache instead of maintaining its own in-process cache for message data? A: It avoids double-buffering (the same data cached twice — once in the JVM heap, once in the OS page cache) and avoids GC pressure from managing a large in-process cache.

Relying on the OS's mature, battle-tested page cache implementation lets Kafka's own memory footprint stay small and predictable, while still getting fast reads for recently-written data that's still page-cached.

Q: What is zero-copy, and how does Kafka use it? A: The `sendfile()` syscall lets the OS transfer bytes directly from a file (or its page cache) to a network socket, entirely in kernel space, without copying through the application's user-space memory. Since consumer fetch requests are often "read a range of an already page-cached log segment," Kafka can serve them via zero-copy, avoiding the cost of copying data into the JVM heap and back out to the network stack — a meaningful throughput win at scale.

Q: Why does Kafka's append-only log design matter for performance, beyond "it's simple"? A: Writes are always sequential appends to the end of the active segment — sequential disk I/O is dramatically faster than random I/O (fewer seeks, better throughput even on SSDs due to more predictable access patterns). It's what makes a distributed log a viable high-throughput design without needing complex in-place-update structures like B-trees, which incur random I/O for updates.

Q: If a consumer is reading data that's fallen out of the OS page cache (e.g. very old messages, or a machine under memory pressure), what happens to read performance? A: The read falls back to actual disk I/O instead of being served from the page cache — meaningfully slower, though still sequential (since it's reading a contiguous range of an append-only segment file) rather than random. This is one reason "how far behind can consumers lag before performance degrades" is a real operational consideration — badly lagging consumers may end up reading cold data from disk instead of hot data from cache.

4. Kafka vs. Traditional Message Queues

Conceptual — a very common interview comparison question ("why Kafka over RabbitMQ/SQS?"), not something this project needed to decide, since Kafka was a given requirement here.

Theory

	Kafka	Traditional MQ (e.g. RabbitMQ)
Consumption model	Pull — consumers call <code>poll()</code> and control their own rate.	Push — broker pushes messages to consumers (with prefetch limits to bound how many are pushed before an ack).
Message lifecycle	Retained per retention policy regardless of consumption; not deleted on read.	Typically removed once acknowledged by its consumer (per queue).

	Kafka	Traditional MQ (e.g. RabbitMQ)
Ordering	Strict per-partition; no ordering across partitions. Enables parallelism with partial (per-key) ordering.	Strict per-queue if single consumer; ordering breaks down with multiple competing consumers on one queue.
Multiple independent consumers of the same stream	Native — separate consumer groups each read the full topic independently, at their own pace.	Requires fanout (e.g. a fanout exchange duplicating the message into multiple queues, one per consumer).
Replay	Natural — reset a consumer group's offset and re-read history (within retention).	Not natural — once consumed/acked, typically gone; some systems support limited replay via separate mechanisms.
Routing complexity	Simple (topic + partition key); complex routing usually built in the application layer.	Rich native routing (exchanges, topic patterns, priority queues) built into the broker.
Best fit	High-throughput event streaming, event sourcing, multiple independent consumers, replay-heavy workloads.	Complex routing, RPC-style request/reply, simpler task-queue workloads.

Q&A

Q: What's the fundamental consumption model difference between Kafka and RabbitMQ?

A: Kafka is pull-based — consumers call `poll()` / `Consume()` and control their own rate, which gives natural backpressure (a slow consumer just polls less often, without the broker needing special handling). RabbitMQ is push-based — the broker pushes messages to consumers as they become available, bounded by a configurable prefetch count to avoid overwhelming a slow consumer, but requiring that tuning to get right.

Q: If two independent applications need to process every event from the same stream, how does this differ between Kafka and a traditional queue?

A: In Kafka this is native: two different consumer groups can independently consume the entire topic at their own pace, since messages aren't deleted on consumption — each group tracks its own offsets. In a traditional queue, a message is normally removed once acknowledged by its single consumer, so you'd typically need a fanout exchange/topic pattern to duplicate the message into two separate queues, one per consuming application.

Q: When would you choose a traditional message queue over Kafka? A: When you need complex routing logic (content-based routing, priority queues, per-message TTL handled natively by the broker) or straightforward RPC-style request/reply task-queue workloads, and don't need replay or multiple independent consumer groups reading the same stream — a traditional MQ's operational model and built-in routing features are often a better fit and simpler to run than a Kafka cluster for that use case.

Q: Why is "ordering" a more nuanced claim for Kafka than for a single traditional queue?

A: A single traditional queue with one consumer gives strict global order, but that serializes processing to one consumer, limiting throughput; adding competing consumers on the same queue breaks that strict order. Kafka's ordering is scoped to a partition — you get strict order *per partition* (and therefore per key, if keyed appropriately), which allows genuine parallelism across partitions while still preserving ordering where it actually matters (per-key), rather than an all-or-nothing tradeoff between throughput and ordering.

5. Producers

Implemented: `KafkaProducer<TKey, TValue>`

(`BuildingBlocks/Kafka/Producer/KafkaProducer.cs`)

Wraps `Confluent.Kafka.IProducer<TKey, TValue>`. Key config (from `KafkaProducerOptions`):

```
Acks = ParseAcks(settings.Acks),                // "All" in our appsettings
EnableIdempotence = settings.EnableIdempotence,    // true
MessageTimeoutMs = settings.MessageTimeoutMs,
RequestTimeoutMs = settings.RequestTimeoutMs,
MessageSendMaxRetries = settings.MessageSendMaxRetries,
RetryBackoffMs = settings.RetryBackoffMs,
CompressionType = ParseCompressionType(settings.CompressionType), // Snappy
BatchSize = settings.BatchSize,
LingerMs = settings.LingerMs,
```

Theory: `acks`

- `acks=0`: fire-and-forget. Producer doesn't wait for any broker acknowledgment. Fastest, least safe — messages can be silently lost.
- `acks=1`: leader acknowledges after writing to its own log, before followers replicate. If the leader crashes right after, before followers catch up, the message is lost.
- `acks=all (-1)`: leader waits for all **in-sync replicas** to acknowledge. Strongest durability. Combined with `min.insync.replicas`, this is what guarantees no data loss on a single broker failure.

We use `acks=All`.

Theory: Idempotent Producer

`EnableIdempotence=true` assigns the producer a **Producer ID (PID)** and attaches a monotonically increasing **sequence number** to each message per partition. The broker deduplicates: if it sees the same PID+sequence number twice (e.g. because the producer retried after a network timeout but the original write actually succeeded), it silently drops the duplicate instead of appending it twice. This solves the classic "retry caused a duplicate" problem *at the broker level*, for a single producer session — it's the foundation exactly-once transactions build on (§9).

Idempotence requires `acks=all`, bounded in-flight requests, and retries enabled — Confluent's client enforces these automatically once you set `EnableIdempotence=true`.

Theory: Batching — `linger.ms` and `batch.size`

Producers don't send one network request per message. They batch messages destined for the same partition and flush a batch when *either* it reaches `batch.size` bytes *or* `linger.ms` has elapsed since the first message in the batch — whichever comes first. Higher `linger.ms` trades a little latency for much better throughput (fewer, bigger requests) and better compression ratios (compressing a bigger batch compresses better).

Theory: The Default Partitioner and Message Keys

When you produce a message with a non-null key, Kafka's default partitioner hashes the key (murmur2 by default) and maps it to a partition via `hash(key) % numPartitions` — same key always maps to the same partition (for a stable partition count). When the key is `null`, the default "sticky partitioner" batches a run of null-key messages onto the same partition for a while (to keep batches efficient) before switching to another partition, rather than strictly round-robinning every single message — an optimization for batching efficiency introduced after earlier Kafka versions did pure round-robin per message.

Q&A

Q: What does `EnableIdempotence=true` actually prevent, precisely? A: Broker-side duplicate appends caused by producer retries within a single producer session — e.g. the producer sends, the ack is lost on the network, the producer retries, but the original write actually succeeded. Without idempotence, that retry creates a duplicate message. It does *not* prevent duplicates from application-level retries (e.g. your own retry-after-failure logic re-calling `Produce` with a "new" logical message) — that's a different problem, solved by idempotent consumers (§13), not the idempotent producer feature.

Q: Why does `acks=all` need `min.insync.replicas` to actually mean anything? A: `acks=all` waits for all *current* ISR members — if ISR has shrunk to just the leader (e.g. all followers are lagging/down),

"all ISR" is satisfied by just the leader acknowledging, which is no safer than `acks=1`.

`min.insync.replicas=2` (with replication factor 3) makes the *produce itself fail* if fewer than 2 replicas are in sync, so you never silently downgrade to single-copy durability.

Q: Why Snappy compression over gzip here? A: Snappy prioritizes speed over compression ratio — good default for latency-sensitive pipelines. Gzip compresses smaller but costs more CPU. LZ4 is often the modern default (fast, decent ratio); zstd gives the best ratio at moderate CPU cost. Choice depends on whether you're CPU-bound or bandwidth-bound.

Q: What's the tradeoff of raising `linger.ms`? A: Throughput and compression efficiency go up (bigger batches = fewer requests, better compression), but per-message latency goes up by up to `linger.ms` in the worst case, since the producer may hold a message waiting for the batch to fill or the timer to expire.

Q: If you produce a message with a `null` key, how does Kafka decide which partition it goes to? A: The default "sticky partitioner" picks a partition and batches a run of null-key messages onto it for efficiency, before periodically switching to a different partition — rather than strictly round-robinning every single message. This is a batching-efficiency optimization; null-key messages have no ordering guarantee tying them to any particular partition regardless.

Q: What happens if a producer sends messages faster than the broker can accept them? A: They queue up in the producer's local buffer (`buffer.memory` / `QueueBufferingMaxKbytes` and `QueueBufferingMaxMessages` in our config). If that buffer fills up before the broker catches up, further `Produce()` calls block (or fail, depending on client configuration/timeout) until space frees up — this is the producer-side backpressure mechanism.

6. Consumers

Implemented: `KafkaConsumer<TKey, TValue>`

(`BuildingBlocks/Kafka/Consumer/KafkaConsumer.cs`)

Key config (from `KafkaConsumerOptions`, `appsettings.json`):

```
"EnableAutoCommit": false,
"EnableAutoOffsetStore": false,
"SessionTimeoutMs": 45000,
"MaxPollIntervalMs": 300000,
"MaxPollRecords": 500,
"PartitionAssignmentStrategy": "CooperativeSticky",
"IsolationLevel": "ReadCommitted"
```

We manually control offset commits — the comment in `KafkaConsumer.cs` explains why:

Offset commit is the handler's responsibility: a transactional handler commits via `SendOffsetsToTransaction`, others call `Commit(message)` explicitly (e.g. after a DLT publish). Auto-committing here would be unsafe for transactional handlers if their transaction was aborted.

Theory: Manual vs. Automatic Offset Commit

`EnableAutoCommit=true` commits offsets periodically in the background, regardless of whether your handler actually finished processing successfully. This can silently lose messages: if the consumer crashes between "offset auto-committed" and "message actually processed," that message is gone from this consumer's perspective on restart. Manual commit — committing only after you've confirmed successful processing (or explicit dead-lettering) — is what gives you **at-least-once** delivery instead of **at-most-once**.

Theory: `isolation.level`

- `read_uncommitted` (default): consumer sees *all* messages, including ones written as part of a transaction that later aborts.
- `read_committed`: consumer only sees messages from committed transactions (plus all non-transactional messages). Messages from an aborted or still-in-flight transaction are invisible until the transaction resolves.

This only matters if *some* producer to that topic uses transactions. Since our

`DeviceEventTransactionalProcessor` writes to `device-events.audit` transactionally, any consumer of that topic should use `read_committed` to avoid seeing partial/aborted data.

Theory: `session.timeout.ms` VS `max.poll.interval.ms`

- `session.timeout.ms`: how long the group coordinator waits without a heartbeat before considering the consumer dead and triggering a rebalance. Heartbeats are sent on a background thread, independent of your poll loop's processing time.

- `max.poll.interval.ms` : how long the coordinator waits between calls to `Consume()` / `poll()` before considering the consumer stuck (e.g. infinite-looping in your handler) and kicking it from the group — *even if heartbeats are still coming in*. This exists because heartbeats alone can't detect "the consumer thread is alive but stuck processing forever."

Theory: Partition Assignment Strategies

- **Range**: assigns contiguous partition ranges per topic to consumers, sorted by consumer ID. Simple but can be uneven across multiple topics.
- **RoundRobin**: spreads partitions round-robin across all consumers, across all subscribed topics. More even, but a full rebalance still reassigns everything from scratch on every membership change.
- **CooperativeSticky** (what we use): incremental rebalancing. On a membership change, only the partitions that *need* to move are revoked/reassigned — everyone else keeps processing uninterrupted. Old "eager" strategies (Range/RoundRobin) revoke *all* partitions from *everyone* first, then reassign — meaning a stop-the-world pause across the whole group on every rebalance, even for a single consumer joining/leaving.

Theory: Consumer Throughput Tuning

- `max.poll.records` (`MaxPollRecords=500` here): caps how many records a single `poll()` call returns — bounds how much work you take on per poll iteration, which in turn bounds how long until your next heartbeat-relevant `poll()` call, relevant to `max.poll.interval.ms`.
- `fetch.min.bytes` : the broker won't respond to a fetch request until at least this many bytes are available (or `fetch.max.wait.ms` elapses) — trades a little latency for fewer, more efficient fetch round-trips under low-throughput conditions.
- `fetch.max.wait.ms` : the maximum time the broker will hold a fetch request open waiting for `fetch.min.bytes` to accumulate, before responding with whatever's available (even if less than the minimum).

Q&A

Q: Why is manual offset commit safer than auto-commit, concretely? A: With auto-commit, offsets advance on a timer regardless of whether your handler succeeded. If the consumer crashes mid-processing after an auto-commit but before finishing the business logic, that message is permanently skipped on restart — silent data loss, at-most-once behavior. Manual commit, done only after confirmed success (or intentional dead-lettering), ensures a crash mid-processing causes redelivery instead of loss — at-least-once.

Q: What's the practical difference between eager and cooperative-sticky rebalancing? A: Eager (Range/RoundRobin) revokes every partition from every consumer in the group first, then reassigns from scratch — a full stop-the-world pause across the whole group for any single membership change. Cooperative-sticky only revokes/reassigns the specific partitions that need to move, letting unaffected consumers keep processing without interruption.

Q: A consumer's heartbeats are healthy but it still gets kicked from the group. Why? A: `max.poll.interval.ms` exceeded — the consumer isn't calling `poll()` / `Consume()` often enough, likely because a handler is stuck or too slow. Heartbeats run on a separate thread and don't reflect whether your processing loop is actually making progress.

Q: If `read_committed` is set but no producer in the system uses transactions, what changes? A: Nothing observable — `read_committed` only filters out messages belonging to open/aborted transactions. With no transactional producer, there's nothing to filter; behavior is identical to `read_uncommitted`.

Q: What's the relationship between `max.poll.records` and `max.poll.interval.ms`? A: `max.poll.records` bounds how much work one `poll()` call hands you; `max.poll.interval.ms` bounds how long you have before calling `poll()` again. If you set `max.poll.records` too high relative to how long your handler takes per record, you risk exceeding `max.poll.interval.ms` while still working through the batch from the previous poll — getting kicked from the group even though you're actively (if slowly) making progress.

Q: What does raising `fetch.min.bytes` trade off? A: Fewer, more efficient fetch requests (better throughput, less per-request overhead) at the cost of added latency under low-throughput conditions, since the broker will wait (up to `fetch.max.wait.ms`) for enough data to accumulate before responding, rather than returning whatever's available immediately.

7. Message Headers, Correlation IDs & Tracing

Implemented

Kafka messages carry optional key-value headers alongside the key/value payload — metadata that doesn't belong in the business payload itself. This project uses them throughout: `event-id`, `event-type`, `correlation-id`, `source`, `schema-version` are attached to every produced message (see `DeviceEventProducer`, `KafkaOutboxMessageFactory`, `DeviceEventTransactionalProcessor`), and the DLT path adds its own diagnostic set: `x-original-topic`, `x-original-partition`, `x-original-offset`, `x-exception-type`, `x-exception-message` (`KafkaDeadLetterProducer`).

Correlation ID propagation is the interesting one: `DeviceEventTransactionalProcessor.ProcessAsync` builds a new, transformed `DeviceEvent` for the audit topic, but deliberately copies the *original* inbound event's correlation ID onto it (`CorrelationId = input.CorrelationId`) rather than generating a new one. This is the seed of distributed tracing — even though the audit event is a technically distinct Kafka message with its own offset and identity, the shared correlation ID lets you reconstruct "everything that happened as a result of this one original business event" across topics and services, just by searching logs/messages for that ID.

Theory: Headers vs. Payload Fields

Headers exist so metadata *about* a message (routing hints, tracing context, content-type, versioning) doesn't have to pollute or be conflated with the actual business payload's schema — and can be read without deserializing the full value. The tradeoff: headers aren't schema-enforced the way the value payload is here (Schema Registry governs the value's shape, not header contents), so header usage is a convention your team has to maintain discipline around, not something the registry will catch drift on.

Q&A

Q: What's the purpose of Kafka message headers, distinct from the key and value? A: They carry metadata about the message that isn't part of the actual business payload — routing/tracing hints, content-type, versioning info, diagnostic context — without polluting or requiring changes to the value's schema, and they're readable without deserializing the full payload.

Q: How does this codebase implement basic distributed tracing across a multi-hop event flow? A: By propagating a `correlation-id` header from the originally-produced event through to derived events — `DeviceEventTransactionalProcessor` explicitly copies `input.CorrelationId` onto the new audit event it produces, rather than generating a fresh one. Even though the audit event is a technically distinct Kafka message with its own offset, the shared correlation ID lets you trace everything that happened as a result of one original business event, across topics.

Q: Why put diagnostic context (`x-exception-type` , `x-original-offset` , etc.) in headers on the DLT message rather than wrapping the payload in an error envelope object? A: Headers keep the original payload byte-for-byte unchanged and immediately replayable back into the source topic once the root cause is fixed, while still carrying full failure context alongside it. An error-envelope wrapper would require unwrapping logic before replay and couples any DLT consumer to a bespoke wrapper schema instead of the original message shape.

Q: This project has both a `schema-version` header and Schema Registry's own schema ID embedded in the wire format — is that redundant? A: Functionally, yes, for enforcement purposes — Schema Registry's embedded schema ID is the actual mechanism that governs deserialization and compatibility checking. The `schema-version` header is informal, human/app-readable

metadata not enforced by anything. Recognizing this kind of "informal parallel mechanism sitting alongside a formal one" is a good design-review instinct — it's not wrong to have both, but it's worth knowing the header isn't doing any enforcement work on its own.

Q: What's a limitation of relying on headers for cross-cutting concerns like tracing, compared to a dedicated tracing system? A: Headers are just conventions — nothing enforces that every producer in a system actually populates them consistently, and there's no built-in correlation/query tooling the way a real distributed tracing system (e.g. OpenTelemetry with a trace backend) provides out of the box. It works well as a lightweight, dependency-free starting point, but doesn't give you trace visualization, span timing, or cross-service dependency graphs the way a dedicated tracing system would.

8. The Outbox Pattern

Theory: The Dual-Write Problem

If application code does `UPDATE business_table; then produce_to_kafka()`, there's no atomicity between the two: the DB commit can succeed while the Kafka produce fails (or vice versa), leaving state and events permanently inconsistent — with no way to roll either back after the fact, since they're two independent systems with no shared transaction.

The outbox pattern solves this by writing the "event to publish" as a row in a DB table, in the **same local transaction** as the business write. Since both writes are now in one DB transaction, they're atomic: both commit or both roll back. A separate background process (the dispatcher) then reads unpublished rows and pushes them to Kafka, retrying independently of the original business transaction. This converts "atomic write across two systems" (hard/impossible without distributed transactions) into "atomic write within one system, plus at-least-once delivery of an already-durable record" (easy).

Implemented

`DeviceEventOutboxService.CreateAsync` (Producer/Services/DeviceEventOutboxService.cs) — business write + outbox insert in one `BeginTransaction / CommitAsync`:

```
await using var transaction = await _dbContext.Database.BeginTransactionAsync(cancellationToken);
try
{
    // Business-specific writes belong here too, same transaction.
    await _repository.AddAsync(outboxMessage, cancellationToken);
    await _dbContext.SaveChangesAsync(cancellationToken);
    await transaction.CommitAsync(cancellationToken);
}
catch { /* rollback */ throw; }
```

KafkaOutboxDispatcher (Producer/Services/KafkaOutboxDispatcher.cs) — a `BackgroundService` polling loop: 1. `RecoverStuckMessagesAsync` — resets rows stuck in `Publishing` past a lease window back to `Failed` (guards against a dispatcher instance crashing mid-publish). 2. `ClaimPendingAsync` — atomically claims a batch:

```
SELECT * FROM kafka_outbox_messages
WHERE status = Pending
      OR (status = Failed AND next_attempt_at_utc <= now)
      OR (status = Publishing AND lease expired)
ORDER BY created_at_utc
LIMIT batchSize
FOR UPDATE SKIP LOCKED
```

1. Publishes each claimed row via the plain (non-transactional) `IKafkaProducer`, then marks it `Published`, or `Failed` with exponential backoff, or `DeadLettered` after `MaxRetryCount` is exhausted.

Theory: `FOR UPDATE SKIP LOCKED`

This is what makes the outbox dispatcher **safe to run as multiple instances** (horizontal scaling / multiple pods). `FOR UPDATE` locks the selected rows within the transaction; `SKIP LOCKED` means a *second* dispatcher instance running the same query concurrently will simply skip rows already locked by the first, instead of blocking and waiting. Each instance ends up claiming a disjoint set of rows — no double-publishing, no lock contention pileup.

Theory: What guarantee does this actually give?

At-least-once delivery to Kafka. The DB row is the durable source of truth; if a publish attempt fails or the dispatcher crashes mid-flight, the row is still there (or gets recovered by the lease mechanism) and will be retried. This means a message *can* be published to Kafka more than once (e.g. if the dispatcher publishes successfully but crashes before marking the row `Published`, it'll retry and publish again on restart) — which is exactly why the *consumer* side needs to be idempotent (§13). The outbox pattern does

not, by itself, give exactly-once — it gives durable at-least-once, and idempotent consumption is what completes the picture.

Q&A

Q: What specific failure does the outbox pattern prevent that a naive "write DB then produce to Kafka" doesn't? A: The DB commit succeeding while the Kafka produce fails (network blip, broker down, process crash between the two calls) — leaving the database updated but no corresponding event ever published, with no way to detect or recover from it after the fact. The outbox makes the "intent to publish" durable and transactional with the business write itself.

Q: Does the outbox pattern give exactly-once delivery? A: No — it gives at-least-once. The dispatcher can crash after a successful Kafka publish but before marking the outbox row `Published`, causing a re-publish on restart. Exactly-once end-to-end requires the *consumer* to be idempotent as well.

Q: Why `FOR UPDATE SKIP LOCKED` instead of just `FOR UPDATE`? A: Plain `FOR UPDATE` would make a second concurrent dispatcher instance *block* waiting for the first instance's transaction to release its locks, then likely re-select the same (now-updated) rows anyway — serializing all dispatcher instances into pointless contention. `SKIP LOCKED` lets each instance immediately grab a different, non-overlapping batch, enabling true horizontal scaling of the dispatcher.

Q: What's the lease/recovery mechanism for, concretely? A: If a dispatcher instance claims a batch (marks rows `Publishing`) and then crashes before finishing, those rows would be stuck in `Publishing` forever without the lease check. `RecoverStuckMessagesAsync` finds rows that have been `Publishing` longer than `LeaseDurationSeconds` and resets them to `Failed` (with an immediate retry), so another dispatcher instance can pick them up.

Q: What's an alternative to a polling dispatcher for reading the outbox table? A: Change Data Capture (CDC) via a tool like Debezium, reading the database's write-ahead log directly instead of polling the table. Lower latency (near-instant vs. poll-interval-bound), no polling load on the DB, but couples you to DB-specific log formats and adds Kafka Connect as an operational dependency. See §21.

Q: What happens to outbox rows that exhaust `MaxRetryCount`? A: They're marked `DeadLettered` in the outbox table itself (a distinct terminal status from the Kafka `DLT` topic used on the consumer side) — a row that's permanently failed to publish and requires manual investigation/intervention, rather than being retried forever.

Q: Why must the outbox insert and the business write share the same database transaction, rather than just being "close together" in the code? A: Atomicity requires them to be indivisible — if a crash happens between two separate transactions (business write committed, outbox insert not yet committed, or vice versa), you're back to the exact dual-write problem the pattern exists to solve. Only a single shared transaction guarantees both happen or neither does.

9. Exactly-Once Semantics & Transactions

Theory: What "Exactly-Once" Actually Means in Kafka

Kafka's EOS doesn't mean "a message is magically delivered exactly once end-to-end across all possible systems." It specifically covers the **consume-transform-produce** loop: consuming from topic A, doing some processing, producing to topic B, and committing the consumer offset for A — all as one atomic unit. Either *all* of {offset commit on A, produce to B} happen, or *none* of them do, even across crashes and retries.

It relies on two building blocks: 1. **Idempotent producer** (§5) — dedup at the broker level per producer session. 2. **Transactions** — group multiple produces (and an offset commit) into one atomic unit across potentially multiple partitions/topics, coordinated by a **Transaction Coordinator** broker role. Each transactional producer has a unique `transactional.id`, a **producer epoch** (bumped on each new session with that ID — this is what fences off zombie instances of the same producer from committing stale transactions), and the coordinator tracks transaction state (`Ongoing` , `PrepareCommit` , `CompleteCommit` , etc.) in an internal `__transaction_state` topic.

Implemented: `KafkaTransactionalProducer<TKey, TValue>`

(`BuildingBlocks/Kafka/Transaction/KafkaTransactionalProducer.cs`)

```
_producer.BeginTransaction();
await produceMessages(); // one or more Produce calls
_producer.SendOffsetsToTransaction(offsetsToCommit, consumerGroupMetadata, timeout);
_producer.CommitTransaction(timeout);
```

Used by `DeviceEventTransactionalProcessor`

(`Consumer/Services/DeviceEventTransactionalProcessor.cs`): consumes a `DeviceEvent` , transforms it, and in one transaction: produces the transformed event to `device-events.audit` **and** commits the consumer's offset on `device-events` via `SendOffsetsToTransaction` . If the process crashes between the produce and the offset commit, on restart the transaction is either fully visible (both happened) or fully rolled back (neither did, and the consumer re-reads from its last committed offset) — no partial state.

Because of this, `DeviceEventConsumer.ProcessMessageAsync` deliberately does **not** call `_consumer.Commit()` on the happy path — the offset commit already happened as part of the Kafka transaction. Calling it again would be redundant, and unsafe if the transaction had actually aborted.

Theory: Producer config for transactions

```
TransactionalId = $"{prefix}-{Guid.NewGuid():N}", // unique per producer instance
EnableIdempotence = true,                        // required for transactions
Acks = Acks.All                                  // required for transactions
```

`InitTransactions()` is called once at producer construction — it registers the `transactional.id` with the coordinator, fences off any previous "zombie" producer instance using the same ID (bumping the epoch), and recovers/completes any transaction left hanging from a previous session.

Theory: The Zombie Producer Scenario, Walked Through

This is worth being able to narrate concretely, not just name-drop "fencing": Suppose a producer instance hangs (e.g. a long GC pause, a network partition) mid-transaction. The orchestration layer (e.g. Kubernetes) decides it's unhealthy and starts a *new* instance with the *same* `transactional.id` (common when `transactional.id` is derived from a stable identity like a pod name or partition assignment, for exactly this restart-recovery reason). The new instance calls `InitTransactions()`, which bumps the producer epoch and tells the coordinator "any transaction from an older epoch on this ID should be considered fenced off." If the *old*, hung instance then wakes up and tries to commit its stale transaction, the coordinator rejects it (epoch mismatch) — preventing the zombie from corrupting state after a new instance has already taken over. Without epoch fencing, the zombie's stale commit could silently interleave with or overwrite the new instance's legitimate work.

Theory: The transaction-poisoning bug (see §26 for the full story)

If a transaction fails **after** `BeginTransaction()` but the code doesn't call `AbortTransaction()` before returning, the underlying producer is left in an "in transaction" state forever — every subsequent `BeginTransaction()` call fails with `Operation not valid in state InTransaction`, permanently breaking that producer instance. This is a real, subtle failure mode worth understanding: **every code path that begins a transaction must guarantee it ends with either commit or abort**, with no way to just "walk away."

Q&A

Q: What does `SendOffsetsToTransaction` actually buy you over calling `Commit()` separately after producing? A: Atomicity across the consume and produce sides. Without it, you'd produce to the output topic, then separately commit the input offset — two independent operations with a crash window between them. If the process dies after the produce but before the offset commit, on restart you'd re-consume and re-produce the same message, duplicating the output. `SendOffsetsToTransaction` makes the

offset commit part of the *same* atomic transaction as the produce, so a crash anywhere in between leaves you with neither-happened, guaranteed re-processed cleanly on restart.

Q: What's a producer epoch and what problem does it solve? A: A monotonically increasing number tied to a `transactional.id`, bumped every time a new producer instance initializes transactions with that ID. It fences out "zombie" producers — e.g. an old instance that hung and is still trying to commit a stale transaction after a new instance has already taken over the same `transactional.id`. The coordinator rejects commits from an old epoch, preventing a zombie from corrupting state after what it thinks is "its" transaction.

Q: Walk through a concrete zombie-producer scenario and how fencing prevents it. A: A producer instance hangs mid-transaction (GC pause, network partition) and the orchestration layer spins up a replacement with the same `transactional.id`. The new instance's `InitTransactions()` call bumps the producer epoch, telling the coordinator to reject anything from the old epoch. If the hung instance later wakes up and tries to commit its stale transaction, the coordinator rejects it due to the epoch mismatch — without this, the zombie could commit stale work concurrently with or after the new instance's legitimate work, corrupting state.

Q: Why does the transactional producer require `EnableIdempotence=true` and `Acks=All`? A: Transactions are built on top of the idempotent producer's dedup mechanism (PID + sequence numbers) — you can't have transactional guarantees without the underlying delivery guarantee idempotence provides. `Acks=All` is required because the coordinator needs certainty that a produce is durably replicated before it can consider that part of the transaction safely committed.

Q: What happens if a consumer with `isolation.level=read_committed` reads a partition where a transaction is still in progress? A: It won't see any messages from that in-progress transaction (or any later messages in the partition, if they'd be out of order relative to the uncommitted ones) until the transaction resolves (commit or abort). This is what gives downstream consumers a consistent view — they never see partial/uncommitted transactional writes.

Q: Is exactly-once semantics free — any tradeoffs? A: Yes, meaningful ones: transactions add coordination overhead (extra round-trips to the transaction coordinator, `SendOffsetsToTransaction` calls), and `read_committed` consumers have to buffer/hide in-flight transactional messages, adding a small latency cost. It also only covers the Kafka-to-Kafka consume-transform-produce loop — if your handler has a *side effect outside Kafka* (e.g. calling an external API), that side effect isn't part of the transaction and can still happen more than once on retry.

Q: What internal topic does the Transaction Coordinator use to track transaction state, and why does that matter? A: `__transaction_state` — an internal Kafka topic (much like `__consumer_offsets`) that durably records each transaction's state (`Ongoing`, `PrepareCommit`, `CompleteCommit`, etc.). It matters because it means transaction state itself survives coordinator broker

failure/failover — a new coordinator can pick up exactly where the old one left off by reading this topic, rather than transaction state being fragile in-memory-only bookkeeping.

10. Kafka Transactions vs. Two-Phase Commit

Conceptual — worth being able to explain precisely why the outbox pattern exists instead of "just using a distributed transaction," since interviewers like probing this.

Theory

Classic **two-phase commit (2PC)** coordinates atomicity across heterogeneous systems (e.g. a database and a message queue) via a global transaction coordinator: a *prepare* phase where all participants vote "can commit" or "must abort," followed by a *commit* phase where the coordinator tells everyone to actually commit (only if all voted yes) or abort. Real-world weaknesses: - **Blocking protocol**: if a participant or the coordinator crashes mid-protocol, other participants can be left holding locks/resources indefinitely in an "in doubt" state, unable to safely resolve on their own. - **Performance cost**: multiple synchronous network round-trips per transaction, with locks/resources held across the whole protocol — expensive at scale. - **Limited support**: most message brokers, including Kafka, don't implement a standard XA/2PC participant interface.

Kafka's own transactions (§9) solve a **narrower** problem than general 2PC: atomicity *within Kafka itself* (multiple produces + an offset commit), coordinated by Kafka's own purpose-built Transaction Coordinator — not atomicity between Kafka and an arbitrary external system like a database. This is precisely why the outbox pattern (§8) exists: rather than attempting distributed-transaction coordination between two fundamentally different systems (a relational database and Kafka), it keeps the atomic operation *within* the database (a single system, using its native transactions) and treats "publish to Kafka" as a separate, independently retryable, at-least-once concern.

Q&A

Q: Why not just use a two-phase commit (2PC) between the database and Kafka instead of the outbox pattern? A: Kafka doesn't provide a standard XA/2PC participant interface, and even where similar coordination exists elsewhere, 2PC is a blocking protocol — a coordinator or participant crash mid-protocol can leave transactions "in doubt," and it requires holding locks/resources across multiple synchronous network round-trips, hurting throughput. The outbox pattern avoids needing distributed-transaction coordination between two heterogeneous systems entirely, by keeping the atomic

operation within a single system (the database) and treating publishing to Kafka as a separate, retryable, at-least-once concern layered on top.

Q: What's the scope of what Kafka's own transactions actually guarantee atomicity over?

A: Only operations within Kafka itself — multiple produces (potentially across partitions/topics) plus a consumer offset commit, coordinated by Kafka's own Transaction Coordinator. It does not extend atomicity to an external system like a database in the same transaction — that's a different problem, solved differently (e.g. the outbox pattern).

Q: What's the core failure mode that makes 2PC risky in practice, distinct from just "it's slow"?

A: Its blocking nature — if the coordinator or a participant crashes between the prepare and commit phases, the remaining participants can be stuck holding resources/locks in an "in doubt" state, unable to independently decide whether to commit or abort without further coordinator input. This can turn a transient failure into an extended outage or manual-intervention scenario, which is a much more serious operational risk than raw latency overhead.

11. Retry Strategy

Implemented: `KafkaRetryExecutor` + `KafkaRetryPolicy` (**BuildingBlocks/Kafka/Retry/**)

```
public bool ShouldRetry(Exception exception) => exception switch
{
    KafkaTransientException => true,
    TimeoutException => true,
    HttpRequestException => true,
    _ => false
};
```

Exponential backoff with a cap:

```
var delay = Math.Min(
    InitialDelayMs * Math.Pow(BackoffMultiplier, attempt - 1),
    MaxDelayMs);
```

Config: `MaxAttempts=3`, `InitialDelayMs=1000`, `BackoffMultiplier=2`, `MaxDelayMs=30000`. After exhausting attempts, throws `KafkaRetryExhaustedException`, which `DeviceEventConsumer` catches and routes to the dead-letter path.

Theory: Transient vs. Permanent Failures

Retrying is only useful for **transient** failures — ones likely to succeed if you just try again (network blip, temporary broker unavailability, a downstream service momentarily overloaded). Retrying a **permanent** failure (a malformed message that will never deserialize, a business rule violation, a bug that deterministically throws) just wastes time and delays the inevitable — and worse, if you retry indefinitely, a single poison message can block an entire partition, since Kafka won't advance past an uncommitted offset.

This is exactly why `KafkaRetryPolicy.ShouldRetry` is a strict allow-list, not "retry everything." Our own `DeviceEventHandler`'s simulated failure (`InvalidOperationException` for `DeviceId==5`) deliberately is *not* in the retry list — it goes straight to dead-lettering.

Theory: Exponential Backoff (and why not fixed-delay retry)

Fixed-delay retry (always wait exactly N ms) risks a "thundering herd" — if a downstream dependency goes down and many callers are all retrying on the same fixed interval, they all hammer it again simultaneously the moment it partially recovers, potentially knocking it back down. Exponential backoff spaces retries increasingly further apart, giving the failing dependency room to actually recover, and a cap (`MaxDelayMs`) prevents backoff from growing unbounded. Production systems often add **jitter** (randomizing the delay slightly) on top, to desynchronize multiple callers retrying the same failure — this codebase's retry policy doesn't include jitter, which is worth knowing as a refinement if asked.

Q&A

Q: Why not just retry every exception type? A: Because most exceptions are permanent (deserialization failures, business logic bugs, validation errors) and retrying them is pure waste — they'll fail identically every time. Worse, since Kafka can't advance the consumer offset past an unprocessed message, endlessly retrying a permanent failure blocks the entire partition from making progress on any *later* message too.

Q: What's the risk of retrying without a cap on delay or attempt count? A: Unbounded delay growth (eventually retries become minutes/hours apart, effectively hanging), and unbounded attempt count means a truly permanent failure never resolves — it just retries forever, blocking the partition indefinitely. Both `MaxDelayMs` and `MaxAttempts` in our policy exist specifically to bound this.

Q: What's jitter and why would you add it? A: Randomizing the retry delay (e.g. actual delay = calculated backoff $\pm$ a random percentage) to prevent many callers that failed at the same moment from retrying in lockstep and re-overwhelming the recovering dependency simultaneously. Our implementation doesn't include it — a reasonable follow-up if asked "how would you improve this."

Q: How does this retry logic interact with Kafka's own consumer-level retry/redelivery? A: They're two different layers. `KafkaRetryExecutor` retries *within a single message's processing*, in-process, before ever giving up. If it exhausts all attempts, the message goes to the DLT and the *original* offset is explicitly committed past (via `_consumer.Commit(message)`) — so Kafka itself never redelivers it. If the retry executor's action throws an exception that *isn't* caught at all (a bug in the exception handling), the base `KafkaConsumer.ConsumeAsync` catches it, logs it, and does *not* commit — which causes Kafka to redeliver the same message on the next poll, a second, coarser layer of "retry."

Q: Why does retrying block the *whole partition*, not just delay that one message? A: Kafka only exposes "commit up to offset N" — there's no way to skip past one message and still process later ones in the same partition while leaving the earlier one uncommitted. Since a consumer processes a partition's messages strictly in order and can't commit an offset ahead of an unresolved one, a message stuck retrying forever holds up every later message behind it in that partition.

12. Dead Letter Topics

Implemented: `KafkaDeadLetterProducer<TKey, TValue>`
(`BuildingBlocks/Kafka/DeadLetter/`)

Publishes the **original raw message** to `{topic}{suffix}` (`device-events.DLT`), with diagnostic headers:

```
["x-original-topic"] = result.Topic,  
["x-original-partition"] = result.Partition.Value.ToString(),  
["x-original-offset"] = result.Offset.Value.ToString(),  
["x-exception-type"] = exception.GetType().FullName,  
["x-exception-message"] = exception.Message
```

`DeviceEventConsumer.DeadLetterAsync` then explicitly commits the consumer offset **past** the poisoned message (`_consumer.Commit(message)`) — treating "successfully published to DLT" as equivalent to "successfully processed," so Kafka doesn't redeliver it and block the partition forever.

Theory: Why a Dead Letter Topic (vs. just logging and dropping, or crashing)

- **Just logging and dropping** loses the message permanently with no way to inspect or replay it later.
- **Crashing the consumer** blocks the entire partition on one poison message — every message behind it in the partition is stuck too, since offsets are strictly ordered.

- **DLT** preserves the message (with failure context) for later inspection/manual replay, *and* lets the consumer keep making progress on subsequent messages. This is the standard pattern for "isolate poison messages without halting the pipeline."

The tradeoff: DLT breaks strict per-partition ordering guarantees for that one message (it's now "out of band"), and someone has to actually own monitoring and acting on the DLT — an unmonitored DLT is just a silent, elegant way to lose data slightly more visibly.

Q&A

Q: Why explicitly commit the offset after dead-lettering, instead of leaving it uncommitted?

A: If left uncommitted, Kafka will redeliver that same poisoned message on the next poll (since the consumer's last committed offset is still before it) — causing an infinite loop of "process, fail, dead-letter, redeliver, process, fail, dead-letter, ..." while making zero progress on any message after it in the partition. Committing past it tells Kafka "this offset is handled," letting the partition proceed.

Q: What's the actual failure mode a DLT protects against that retry alone doesn't? **A:** A message that is *permanently* unprocessable (bad data, a bug that always throws for this specific input). Retry handles *transient* failures; once retries are exhausted (or the failure is immediately classified as non-retryable), something still has to happen to that message so it doesn't block the partition forever — that's the DLT's job.

Q: What operational practice does a DLT require to actually be useful? **A:** Active monitoring and alerting on DLT volume/contents, plus a defined process for triage — manual fix-and-replay, automated reprocessing after a bug fix, or deliberate discard. A DLT nobody watches is equivalent to silently dropping messages, just with better paperwork.

Q: Why publish the *original raw message* to the DLT rather than some wrapped/transformed error object? **A:** Preserving the exact original bytes/message means you can potentially replay it back into the original topic unmodified once the root cause is fixed, without needing to reconstruct or guess at the original payload. The diagnostic headers (`x-exception-type` , `x-original-offset` , etc.) provide the failure context alongside it without touching the original data.

Q: What would you need to build to actually "replay" a DLT message back into the source topic? **A:** A small tool/process that reads from the DLT topic, strips the diagnostic headers (or leaves them, depending on whether the source topic's consumers care), and republishes the payload to the original topic — typically gated behind a manual trigger or approval step, since blindly auto-replaying could reintroduce the same failure in a loop if the root cause wasn't actually fixed.

13. Idempotent Consumers

Implemented: `InMemoryProcessedEventStore` (**Consumer/Idempotency/**)

```
private readonly HashSet<string> _processedEvents = new();  
// HasProcessedAsync(eventId) checks membership; MarkProcessedAsync adds it.
```

`DeviceEventHandler.HandleAsync` checks this *before* doing any business processing, and returns early (no-op) if the event was already processed.

Theory: Why idempotent consumption is required, not optional

Combine two facts already covered: (1) the outbox dispatcher gives **at-least-once** delivery to Kafka — meaning the same logical event *can* land on the topic more than once; (2) manual offset commit after processing means a crash between "processing done" and "offset committed" causes Kafka to **redeliver** that message. Both are legitimate, expected behaviors of an at-least-once system — not bugs. The only way to get a net *exactly-once effect* on the consumer's business state, given an at-least-once delivery mechanism, is for the consumer's processing itself to be **idempotent**: processing the same message twice must produce the same end state as processing it once.

Theory: Dedup strategies, and why "in-memory" is a demo shortcut

- **In-memory set** (what we built): fast, zero infrastructure — but the dedup history is lost on every restart, so a redelivered message after a restart is *not* caught. Only correct for demonstrating the *pattern*, not for production.
- **Redis with TTL**: shared across consumer instances, survives individual instance restarts, and a TTL bounds memory growth (you don't need to remember an event ID forever — just long enough to cover the realistic redelivery window).
- **Database unique constraint**: e.g. a `processed_events(event_id PRIMARY KEY)` table; insert-or-ignore semantics naturally dedup, and if the business write and the dedup-marker write are in the *same* DB transaction, you get atomicity between "did the business effect happen" and "is this marked processed" — closing a subtle window that Redis (a separate system from your business DB) can't close as cleanly.

Theory: The Zombie Consumer Connection

Idempotency isn't just about crashes — it's also what protects you during a **zombie consumer** rebalance scenario (§18): a consumer that appears dead to the group coordinator (e.g. a long GC pause) can have its partitions reassigned to another consumer, then wake up and finish processing its last-

fetches batch anyway. Both the "zombie" and the new partition owner can end up processing overlapping messages concurrently. Idempotent processing means this results in redundant work, not corrupted state.

Q&A

Q: Why is the in-memory `HashSet` dedup store explicitly *not* production-ready? A: It's per-process, in-memory state — a restart (deploy, crash, scale event) wipes it clean. Any message redelivered after a restart, but already processed before that restart, would not be recognized as a duplicate and would be processed again. It's fine for demonstrating the *pattern* but not a durable idempotency guarantee.

Q: What's the advantage of a DB-unique-constraint dedup approach over Redis? A: If the dedup marker insert happens in the *same transaction* as the business state change, you get atomicity: either both the business effect and the "processed" marker commit together, or neither does. With Redis (a separate system), there's an unavoidable window where the business change commits but the Redis marker write fails (or vice versa) — a smaller-scale version of the same dual-write problem the outbox pattern solves on the producer side.

Q: Does idempotent consumption alone solve everything about exactly-once processing? A: No — it solves duplicate *delivery*, not duplicate *side effects outside your own dedup boundary*. If your handler calls an external API as part of processing, and it crashes after the API call but before marking the event processed, a redelivery would call that external API again too — unless the external call itself is also idempotent (e.g. using an idempotency key the external system understands).

Q: How would you size a TTL for a Redis-based dedup store? A: Base it on the realistic maximum redelivery window for your system — e.g. how long a message could plausibly sit unprocessed/uncommitted before being redelivered (bounded by things like `max.poll.interval.ms`, retry backoff ceilings, and how long you'd tolerate a stuck consumer before manual intervention). It needs to comfortably exceed that window, with margin, or you risk the dedup record expiring before a legitimate redelivery arrives.

Q: How does idempotent consumption relate to the "zombie consumer" rebalance scenario? A: A consumer that's timed out (from the coordinator's perspective) but actually still alive can keep processing its last-fetched batch after its partitions have already been reassigned — meaning both it and the new owner could process overlapping messages concurrently. Idempotent processing turns that overlap into merely redundant work (safe) instead of corrupted or duplicated business state (unsafe) — it's another reason idempotency is treated as a baseline requirement, not an edge-case nicety.

14. Large Messages & the Claim-Check Pattern

Conceptual — this project's messages are small JSON payloads, so this never came up in practice, but it's a common practical question about Kafka's real-world limits.

Theory

Kafka has a configurable max message size (`message.max.bytes` on the broker/topic, `max.request.size` on the producer) — historically defaulting to around 1MB, though commonly raised in practice. Sending very large payloads (large files, big blobs, images) directly through Kafka as message values is a well-known anti-pattern: it doesn't just cost more for that one message — it degrades replication traffic, page cache efficiency, and batching effectiveness for *every* message sharing that broker/topic/partition, not just the large ones.

The claim-check pattern: store the large payload in external blob storage (S3, Azure Blob Storage, etc.), and publish only a lightweight reference — a "claim check" (a URL or object key) — as the actual Kafka message. Consumers fetch the real payload from blob storage using that reference only when/if they actually need it.

Q&A

Q: What's the risk of sending large payloads (multi-MB files) directly as Kafka message values? A: It degrades performance broadly, not just for that message — bigger messages mean more replication network traffic, more page cache pressure, and work against effective batching (a producer's batch fills up faster with large messages, and consuming/processing large messages takes proportionally longer, reducing overall throughput for the partition/topic they share with other, smaller messages).

Q: What's the claim-check pattern and when would you use it? A: Store the actual large payload in external blob storage and publish only a lightweight reference (a URL/object key) to Kafka. Consumers fetch the real content from blob storage using that reference when needed. Use it whenever payloads regularly exceed a few hundred KB to low single-digit MB — it keeps Kafka's message size small and predictable while still supporting large-payload workflows.

Q: What's a downside of the claim-check pattern compared to just raising Kafka's max message size? A: Added complexity and a new failure mode — the blob storage reference and the actual blob can now get out of sync (e.g. the blob is deleted or not yet uploaded when the Kafka message is consumed), and consumers now depend on the availability of a second system (blob storage) to fully process a message, not just Kafka. It's the right tradeoff for genuinely large or infrequent large payloads, but it's not "free" compared to simply keeping messages small in the first place.

15. Schema Registry & Schema Evolution

This is the deepest area of this project — we went well past "textbook compatibility mode definitions" into actual SDK behavior. Take your time here; it's a strong differentiator if you can speak to it fluently.

Theory: Why a Schema Registry

Without one, producers and consumers agree on message shape only by convention/documentation — any drift (a renamed field, a changed type) breaks consumers silently or loudly, discovered only at runtime. A Schema Registry makes the contract explicit and enforced: producers register a schema per **subject** (by convention `{topic}-value` / `{topic}-key`); the registry enforces a **compatibility mode** so an incompatible schema change is rejected *at publish time*, not discovered later by a crashing consumer in production.

Wire format: 1 magic byte + 4-byte schema ID (big-endian) + the actual payload. Consumers read the ID, fetch the corresponding schema from the registry (cached after first fetch), and use it to interpret the bytes.

Implemented: The serialization path

- `KafkaSchemaSerializerFactory` / `KafkaSchemaDeserializerFactory`
(`BuildingBlocks/Kafka/SchemaRegistry/`) wrap
`Confluent.SchemaRegistry.Serdes.Json.JsonSerializer<T>` / `JsonDeserializer<T>`.
- Config: `AutoRegisterSchemas` (register a new version automatically on first produce with a changed shape) and `UseLatestVersion`.
- A key async/sync SDK constraint we hit: `Confluent.Kafka.ProducerBuilder` supports both sync (`ISerializer<T>`) and async (`IAsyncSerializer<T>`) value serializers — schema-registry serializers are async. But `ConsumerBuilder` **only** supports the synchronous `IDeserializer<T>` — there's no async overload on the consume side. We bridge this with `Confluent.Kafka.SyncOverAsync`'s `.AsSyncOverAsync()` extension, wrapping the async `JsonDeserializer<T>` into a sync-compatible shape.

Theory: Compatibility Modes

Mode	Guarantee	Typical use
BACKWARD	New schema can read data written with the <i>previous</i> schema. Consumers upgrade first.	Default in most orgs — evolve consumers ahead of producers.
BACKWARD_TRANSITIVE	New schema can read data from <i>all</i> previous schema versions, not just the immediately preceding one.	Stronger guarantee when very old data might still be read.
FORWARD	Data written with the <i>new</i> schema can still be read by the <i>previous</i> schema. Producers upgrade first.	When you must roll out producers before consumers.
FORWARD_TRANSITIVE	Same, but against all future consumers reading with any older schema, transitively.	Rare; strong forward guarantee.
FULL	Both BACKWARD and FORWARD must hold.	Safest, most restrictive — usually only additive optional-field changes survive.
FULL_TRANSITIVE	FULL, transitively across all versions.	Maximum safety, maximum restriction.
NONE	No compatibility checking at all.	Emergency/manual override only — used ourselves temporarily to clear a self-inflicted blocked transition during testing; not a normal operating mode.

The empirical finding that most engineers don't know: open vs. closed content model

JSON Schema's `additionalProperties: false` ("closed content model" — no properties beyond what's declared are allowed) vs. its absence/ `true` ("open content model") **inverts which compatibility direction favors additive field changes**:

- **Closed model**: adding a new **optional** field passes **BACKWARD** (new schema, missing that field in old data, is fine since it's optional) but **fails FORWARD/FULL** — because the *old* schema, being closed, would reject a message that now includes an undeclared field.
- **Open model**: adding a field **passes FORWARD** (old schema doesn't declare it, but being open, doesn't reject the extra field either) but **fails BACKWARD** — Confluent's specific JSON-schema diff algorithm flags `PROPERTY_ADDED_TO_OPEN_CONTENT_MODEL` even for genuinely optional fields,

reasoning that the *reader* schema (whichever side is "reading" in that direction) declaring a property the *writer* schema doesn't have is inherently risky.

- **Removing a field fails under every mode**, regardless of open/closed — there's no compatibility mode that makes deletion of a field safe. The correct practice is deprecating a field (stop using it, keep it nullable/present) rather than removing it from the schema.

We verified all of this empirically against a live Schema Registry using the `POST /compatibility/subjects/{subject}/versions/latest` dry-run endpoint — see §26 for the exact test matrix and results.

The deeper SDK gotcha: what a consumer actually validates against

Reading Confluent's actual `JsonDeserializer<T>.DeserializeAsync` source revealed something non-obvious: after deserializing the incoming bytes into your CLR type `T`, the deserializer **re-serializes** that object and validates the round-tripped JSON against the subject's **latest registered schema in the registry** — not the schema the message was originally written with, and not just your local C# type in isolation. Practical consequence: **if you add a field to your consumer's type but no producer has registered a schema version including that field yet, the consumer's own re-serialized object (now including the new field, e.g. as an explicit `null`) gets rejected by the still-old, still-closed registered schema** — even for data the consumer successfully deserialized moments earlier. Every producer to a topic must register its schema before a consumer relying on a newly-added field can process *anything* from that topic, old or new.

Theory: Subject Naming Strategies

- **TopicNameStrategy** (default, what we use): subject = `{topic}-value` (and `{topic}-key`). One schema per topic — simplest, but means a topic can only ever carry one message type.
- **RecordNameStrategy**: subject = the fully-qualified record/message type name, independent of topic. Allows multiple message types on the same topic, each evolving independently.
- **TopicRecordNameStrategy**: combines both — `{topic}-{record-name}` — multiple types per topic, but still scoped per-topic.

Theory: Avro vs. JSON Schema vs. Protobuf

	Wire size	Schema evolution	Ecosystem	Notes
Avro	Compact binary	Mature writer/reader schema resolution (a reader can read data from a different but "resolvable" writer schema, filling in defaults/dropping unknowns automatically)	Most common in Kafka-native shops	No field names in the payload itself — needs the schema to even parse

	Wire size	Schema evolution	Ecosystem	Notes
JSON Schema	Larger (text), or compact if payload itself is JSON	Compatibility rules as covered above; validation is more "check the whole object against a big rule set" than "reader/writer resolution"	Human-readable, easy to debug on the wire (once you strip the 5-byte header)	What we used here
Protobuf	Compact binary	Field-number-based evolution (fields identified by number, not name/position — very old and new schemas can usually coexist by design)	Common where gRPC is already in use	Requires <code>.proto</code> definitions and codegen

Theory: Avro's Reader/Writer Schema Resolution, In a Bit More Depth

Since Avro is the most common production choice and interviewers often probe it specifically: Avro readers don't validate against "the latest schema" the way we saw JSON Schema do — instead, a reader has its *own* schema (the one its code was compiled against), and Avro's resolution rules reconcile it against whatever *writer* schema the data was actually encoded with (identified by the schema ID, same wire-format idea as JSON Schema here). If the reader schema has a field the writer schema doesn't, Avro fills it with that field's declared **default value** (this is why Avro effectively *requires* defaults for safely-added fields, more explicitly than JSON Schema's "just make it optional"). If the writer schema has a field the reader schema doesn't, Avro simply drops it during resolution. This reader/writer resolution model is a fundamentally different (and, many argue, more predictable) mechanism than JSON Schema's "re-serialize and validate against a big rule set" approach we found via SDK source-reading in this project.

Q&A

Q: Explain BACKWARD vs. FORWARD compatibility in one sentence each, correctly. A: BACKWARD — a schema change is safe if new-schema *consumers* can still read data written by *old-schema* producers. FORWARD — a schema change is safe if old-schema *consumers* can still read data written by new-schema *producers*.

Q: You add an optional field to a closed-model (`additionalProperties: false`) JSON schema. Which compatibility mode(s) does it pass, and why? A: BACKWARD passes — a new consumer just finds the field absent in old data, which is fine since it's optional. FORWARD (and therefore FULL) fails — the *old*, closed-model schema would reject a message that now carries an undeclared extra property, since closed content models disallow anything not explicitly listed.

Q: Is removing a field from a schema ever safe under any compatibility mode? A: No — deletion breaks compatibility in every mode we tested (BACKWARD, FORWARD, FULL), regardless of

open or closed content model. The standard practice is to deprecate the field (stop populating it, but keep it declared/nullable) rather than remove it outright.

Q: What's the practical operational implication of "consumers validate against the subject's latest registered schema, not the schema the message was written with"? A:

Rollout order matters more than you'd naively assume. Even if you're only *adding* an optional field (theoretically safe), a consumer with an updated type can fail to process *any* message — old or new — if no producer has registered a schema version including that field yet, because the consumer's own post-deserialization validation step checks against whatever's currently "latest" in the registry, which is still the old, narrower schema.

Q: What does `AutoRegisterSchemas` control, and what's a production alternative? A: Whether the producer automatically registers a new schema version to the registry the first time it produces a shape the registry hasn't seen. Convenient for development, but risky in production — an accidental type change ships a new schema version live, with no review step. Production teams often disable it and register schemas explicitly as a reviewed CI/CD step (schema files as source-controlled artifacts), with the app configured to `UseLatestVersion` instead.

Q: When would you pick Avro over JSON Schema for a new system? A: When wire size and parsing performance matter (Avro's compact binary format beats JSON's text overhead), and when you're in a Kafka-native ecosystem where Avro's writer/reader schema resolution tooling is already well-supported. JSON Schema is a reasonable choice when human-readability of the payload (debugging, ad-hoc tooling) matters more than raw efficiency.

Q: What subject naming strategy would you use for a topic carrying multiple distinct event types? A: `RecordNameStrategy` or `TopicRecordNameStrategy` — `TopicNameStrategy` (the default, one schema per topic) can't represent more than one message shape per topic at all.

Q: How does Avro's reader/writer schema resolution handle a field the reader expects but the writer's data doesn't have? A: It fills in that field's declared default value during resolution — which is why Avro effectively requires a default value to be specified for any field you want to safely add later, more explicitly than JSON Schema's looser "just mark it optional/nullable" convention.

Q: What happens under Avro resolution if the writer's data has a field the reader's schema doesn't know about? A: It's simply dropped during resolution — the reader never sees it, with no error. This is part of why Avro's evolution model is often considered more predictable than JSON Schema's rule-based validation approach: the resolution behavior for both "reader has extra field" and "writer has extra field" is explicit and well-defined by the schema resolution algorithm itself, rather than depending on content-model settings like `additionalProperties`.

16. Delivery Semantics — The Full Picture

Theory

- **At-most-once:** a message is delivered zero or one times — never redelivered, but can be lost. Typically the result of committing offsets *before* processing (or auto-commit racing ahead of actual processing).
- **At-least-once:** a message is delivered one or more times — never silently lost, but can be duplicated. Result of committing offsets *after* confirmed processing; a crash between processing and commit causes redelivery.
- **Exactly-once:** the *effect* of processing happens exactly once, even though delivery/redelivery can still occur underneath. Achieved either via Kafka transactions (for Kafka-to-Kafka consume-transform-produce, §9) or via at-least-once delivery + idempotent processing (§13) — these are the two paths to the same outcome, and it's worth being able to name both.

Mapped to what we actually built

Component	Semantics	Why
Outbox dispatcher → <code>device-events</code>	At-least-once	DB row is durable; a crash after publish-but-before-marking-published causes a republish.
<code>DeviceEventHandler</code> business logic	Effectively exactly-once	At-least-once delivery + <code>InMemoryProcessedEventStore</code> idempotency check = exactly-once <i>effect</i> (with the caveat that the in-memory store isn't durable — see §13).
<code>DeviceEventTransactionalProcessor</code> → <code>device-events.audit</code> + offset commit	Exactly-once (true Kafka EOS)	<code>SendOffsetsToTransaction</code> + <code>CommitTransaction</code> atomically ties the produce and the offset commit together.
Dead-lettering path	At-least-once semantics preserved	Original message preserved verbatim; offset explicitly committed past it so the partition isn't blocked.

Q&A

Q: Name the three Kafka delivery semantics and how each is typically achieved. A: At-most-once — commit offset before/without confirming processing (or rely on auto-commit's timing). At-least-once — commit offset only after confirmed processing, accepting that a crash mid-way causes redelivery. Exactly-once — either Kafka transactions (for consume-transform-produce loops) or at-least-once delivery combined with idempotent processing.

Q: In this codebase, is the *end-to-end* pipeline (DB write → Kafka → business effect) *exactly-once*, or something else? A: It's at-least-once delivery end-to-end (the outbox can republish, and message redelivery is possible on consumer crash), combined with idempotent processing at the point where business effects actually happen — which nets out to an *effectively* exactly-once outcome, without claiming true Kafka-transactional exactly-once for the whole pipeline (that's only true for the narrower consume-transform-produce step inside the consumer).

Q: Why is "exactly-once" a slightly misleading term? A: Because it doesn't mean a message is physically delivered exactly one time at the network/protocol level — duplicates and redeliveries absolutely still happen underneath. It means the observable *effect* of processing is as if it happened exactly once, achieved either by deduping (idempotent consumption) or by atomic all-or-nothing produce+commit (transactions). Explaining it this way in an interview signals you actually understand the mechanism, not just the marketing term.

Q: Which delivery semantic would you pick for a use case where losing a message is acceptable but duplicate processing is not (e.g. sending a one-time promotional email)? A: At-most-once — commit the offset before or without confirming full processing, accepting the small risk of losing a message on a crash, specifically to guarantee you never send the same email twice. This is a case where the "safer-sounding" at-least-once is actually the wrong choice unless paired with idempotent processing (e.g. an idempotency key the email system understands) — otherwise duplicate sends are the worse failure mode here.

17. Partitioning & Ordering (Conceptual)

Not implemented in this solution — every topic here runs with Kafka's single-partition default. This section is pure theory, worth knowing at this experience level even without hands-on practice here.

Theory

- Kafka guarantees ordering **only within a single partition**. Across partitions, there's no ordering guarantee at all — messages produced to partition 0 and partition 1 can be consumed in any relative order.
- The **partition key** (in our code, `deviceEvent.DeviceId.ToString()`, used as the Kafka message key) determines which partition a message goes to, via a hash of the key modulo partition count (by default). Same key → same partition, every time (as long as partition count doesn't change) → strict ordering for that key's messages.

- **Consequence:** if you need "all events for device X processed in order," keying by device ID and relying on same-partition ordering is correct *only as long as partition count is stable*. Changing partition count reshuffles the key→partition mapping for *all* keys, breaking the "same key always same partition" assumption for existing data (though new messages after the change are internally consistent again).
- **Partition count also caps consumer parallelism** within a group (§6) — more partitions means more possible concurrent consumers, at the cost of weaker ordering scope (ordering is per-partition, so more partitions = ordering guaranteed over a narrower slice of the keyspace, assuming a reasonable hash distribution).
- Choosing partition count is a one-time-ish decision with real consequences: too few limits scale-out; too many adds per-partition overhead (open file handles, replication traffic, more overhead during rebalances) and can be costly to reduce later (Kafka doesn't support shrinking partition count on an existing topic — only growing it).

Q&A

Q: If `DeviceEvent` messages are keyed by `DeviceId`, what ordering guarantee do you actually get? A: All events for the same `DeviceId` land on the same partition (given stable partition count) and are consumed in the order they were produced, since ordering is guaranteed per-partition. Events for *different* device IDs have no ordering relationship to each other at all — they may well be interleaved across different partitions and consumed in any relative order.

Q: What happens to existing ordering guarantees if you increase a topic's partition count?

A: The key→partition hash mapping changes for (potentially) every key, since it depends on partition count. Historical messages stay in their original partitions, but new messages for the same key may now land on a *different* partition than the old ones did — meaning "all of device X's history in one ordered partition" no longer holds across the partition-count change. New messages going forward are internally consistent again, just potentially in a different partition than before.

Q: Can you shrink a Kafka topic's partition count? A: No — Kafka doesn't support reducing partition count on an existing topic (doing so would require redistributing/discarding data with no clean semantics). If you over-provisioned partitions, the practical fix is usually creating a new topic with the right count and migrating.

Q: What's the tradeoff of choosing a very high partition count "to be safe"? A: More partitions means more open file handles and metadata overhead per broker, more replication traffic, slower/heavier rebalances (each partition is a unit of reassignment work), and longer leader-election recovery on broker failure (more partitions to elect leaders for). It's not free "just in case" scaling headroom — it has ongoing operational cost.

Q: If a single key (e.g. one extremely active `deviceId`) produces far more traffic than all others, what problem does that create? A: A "hot partition" — since that key always maps to the same partition, all of its traffic concentrates on one partition regardless of how many total partitions the topic has, creating an uneven load that adding more partitions doesn't fix (the hot key still only uses one of them). Mitigations include splitting an unusually hot logical key into sub-keys (e.g. appending a bucket suffix) if strict per-key ordering across the *entire* hot key isn't actually required, or accepting the skew if it is.

18. Consumer Group Rebalancing (Conceptual)

Partially covered in §6 (CooperativeSticky vs. eager) — this section goes one level deeper into triggers and mechanics.

Theory

What triggers a rebalance: - A consumer joins or leaves the group (including a graceful shutdown). - A consumer is considered dead by the group coordinator — either `session.timeout.ms` elapses without a heartbeat, or `max.poll.interval.ms` elapses without a `poll()` call. - Partition count on a subscribed topic changes. - A consumer explicitly calls an unsubscribe/resubscribe.

Eager rebalancing (older protocol, Range/RoundRobin strategies): on any trigger, *every* consumer in the group revokes *all* its partitions, the coordinator computes a fresh assignment from scratch, and partitions are reassigned — meaning the entire group stops processing during the rebalance window, even if only one consumer's membership actually changed.

Cooperative (incremental) rebalancing (CooperativeSticky): only the specific partitions that actually need to move are revoked and reassigned; consumers whose partition assignment doesn't change keep processing uninterrupted throughout. This can take more than one rebalance round-trip to converge but avoids the stop-the-world pause.

Static group membership (`group.instance.id`): assigns a consumer a stable identity across restarts. A consumer restarting within `session.timeout.ms` with the same static ID is treated as "the same member returning," skipping a full rebalance entirely — useful for reducing rebalance churn during rolling deploys.

Theory: The Zombie Consumer Scenario

A consumer can appear dead to the group coordinator (missed `session.timeout.ms`, e.g. due to a long GC pause or transient network partition) and have its partitions reassigned to another consumer — while actually still being alive, and finishing processing of its last-fetched batch anyway. Both the "zombie" and the new partition owner can then end up processing (and potentially producing side effects for) overlapping messages concurrently. This is exactly the kind of scenario that makes idempotent processing (§13) and fencing mechanisms (like Kafka transactions' producer epochs, §9) necessary as a baseline design requirement, not an edge-case nicety — "the old owner definitely stopped before the new owner started" is not a safe assumption to build on.

Q&A

Q: List at least three distinct triggers for a consumer group rebalance. A: A consumer joining or leaving the group; a consumer being timed out (session timeout or max poll interval exceeded); a change in partition count on a subscribed topic.

Q: Why does `max.poll.interval.ms` exist as a *separate* timeout from `session.timeout.ms` ? A: Heartbeats run on a background thread independent of your processing loop — a consumer can be sending healthy heartbeats while its main thread is stuck in an infinite loop or an extremely slow handler, never calling `poll()` again. `max.poll.interval.ms` specifically detects "the processing loop itself is stuck," which heartbeats alone can't reveal.

Q: What's the practical benefit of static group membership during a rolling deployment? A: Without it, every pod restart during a rolling deploy triggers a full rebalance (the old member leaves, a "new" member joins). With a stable `group.instance.id` and a restart fast enough to stay within `session.timeout.ms`, the coordinator recognizes it as the same member returning and skips the rebalance — meaningfully reducing rebalance churn (and the associated processing pause) during routine deploys.

Q: What's a "zombie consumer" and why is it dangerous during a rebalance? A: A consumer that appears dead to the group coordinator (e.g. missed its session timeout due to a long GC pause or network blip) and has its partitions reassigned to another consumer — but is actually still alive and finishes processing its last-fetched batch afterward. Both the zombie and the new owner can end up processing/producing side effects for overlapping messages concurrently, which is why idempotent processing and fencing mechanisms are treated as baseline requirements rather than optional hardening.

Q: What's the "generation ID" in the context of consumer group coordination, at a conceptual level? A: A monotonically increasing number the coordinator bumps every time group membership changes (a rebalance occurs). It lets the coordinator (and consumers) detect stale requests — e.g. if a consumer tries to commit an offset tagged with an old generation ID after a rebalance has

already moved that partition elsewhere, the coordinator can reject it as stale, similar in spirit to how producer epochs fence off zombie producers (§9).

19. Cluster, Replication & Broker Internals (Conceptual)

Not implemented — this project runs a single-broker KRaft setup throughout (`docker-compose.yml` 's `kafka` service, `apache/kafka:latest` , one node). No replication, no multi-broker failure scenarios were exercised. Know this conceptually.

Theory

- **Replication factor:** how many copies of each partition exist across brokers. $RF=3$ is a common production default — tolerates one broker failure without data loss (with `min.insync.replicas=2`) and often two without *availability* loss depending on configuration.
- **Leader/follower:** one replica per partition is the leader (all client reads/writes go through it); the rest are followers, continuously replicating the leader's log.
- **ISR (in-sync replicas):** followers that are caught up within a configurable lag threshold. Only ISR members are eligible for leader election if the current leader fails — this is what prevents "the new leader is a stale, lagging replica," which would silently lose recently-written data.
- `min.insync.replicas` : the minimum ISR size required for a produce with `acks=all` to succeed. With $RF=3$ and `min.insync.replicas=2` , you can lose one broker and keep operating normally; losing a second broker (dropping ISR below 2) makes further `acks=all` produces fail outright rather than silently accepting weaker durability.
- **KRaft vs. ZooKeeper:** KRaft (Kafka Raft metadata mode, what this project uses) is the modern, ZooKeeper-free architecture — Kafka brokers themselves (a subset acting as "controllers") manage cluster metadata via the Raft consensus protocol, removing the separate ZooKeeper dependency entirely. ZooKeeper mode is deprecated as of Kafka 4.0.
- **Log segments:** a partition's log isn't one giant file — it's split into segment files, rolled over by size or time. Retention/compaction operate at the segment level (a whole segment is deleted/compacted once eligible), not per-message.
- **Unclean leader election** (`unclean.leader.election.enable`): if *no* ISR replica is available (all have failed or fallen too far behind), this setting controls whether Kafka is allowed to elect a non-ISR (out-of-sync) replica as leader anyway, trading consistency for availability — off by default in modern Kafka, since it can silently lose data, but it's a real, named tradeoff worth knowing exists.

Q&A

Q: With replication factor 3 and `min.insync.replicas=2` , how many broker failures can you tolerate before writes start failing? A: One broker failure keeps you at 2 in-sync replicas — still meets `min.insync.replicas` , so `acks=all` produces keep succeeding. A second simultaneous failure drops you to 1 in-sync replica, below the minimum, so further `acks=all` produces fail outright (by design — better a loud failure than silently weakening the durability guarantee).

Q: Why must a new partition leader be elected from the ISR specifically, not just any replica? A: A replica outside the ISR is, by definition, lagging behind the leader's log — it doesn't have the most recent writes. Electing it as leader would silently lose those recent writes (any consumer/producer would now see an older state as if it were current). Restricting leader election to ISR members guarantees the new leader has everything that was acknowledged under `acks=all` .

Q: What's the core architectural difference between KRaft and ZooKeeper-based Kafka? A: ZooKeeper mode uses a separate ZooKeeper ensemble as the external system of record for cluster metadata (broker list, partition assignments, ISR state, etc.). KRaft removes that external dependency — a subset of the Kafka brokers themselves act as controllers and manage that same metadata internally via the Raft consensus protocol, simplifying the deployment (one system instead of two) and improving controller failover time.

Q: What's a log segment, and why does retention operate on segments rather than individual messages? A: A partition's log is physically stored as a series of segment files, rolled over by size/time. Retention and compaction act on whole segments (delete/compact an entire eligible segment file) rather than scanning and removing individual messages — far more efficient, since it avoids rewriting large files to remove a few old entries; instead, whole (already-immutable) segment files are just deleted once entirely past the retention window.

Q: What is unclean leader election, and why is it disabled by default? A: It's the option to elect a non-ISR (out-of-sync, lagging) replica as partition leader when no ISR replica is available at all — trading data consistency for availability (the partition can keep accepting writes/serving reads instead of being unavailable, but the new leader may be missing recently-acknowledged writes). It's disabled by default because silently losing acknowledged data is usually considered worse than a temporary availability gap, though some availability-prioritizing workloads deliberately enable it.

20. Kafka Streams & ksqlDB (Conceptual)

Not implemented — this project uses plain producer/consumer clients throughout, not the Streams library. Worth knowing conceptually, especially since our hand-rolled `DeviceEventTransactionalProcessor` (consume, transform, produce) is exactly the pattern Kafka Streams exists to make easier.

Theory

Kafka Streams is a Java/Scala library (runs as a normal application process, not a separate cluster service) for building stream-processing applications directly on top of Kafka topics.

- **KStream**: represents an unbounded stream of individual, independent events — every record is a distinct fact.
- **KTable**: represents a continuously-updated table — the *latest value per key* — conceptually similar to a compacted topic's view, where a new record for an existing key is interpreted as an update to that key's current value rather than an independent new fact. The same underlying topic can often be interpreted as either a KStream or a KTable depending on what you're modeling.
- **Stateful operations** (aggregations, joins, windowing) are backed by local **state stores** (RocksDB by default), which are themselves backed by a Kafka **changelog topic** for fault tolerance — if a Streams instance crashes or its partitions are reassigned, the relevant state can be rebuilt by replaying that changelog topic, without needing external distributed storage.
- **Windowing**: grouping stream events into time-bounded windows (tumbling, hopping, sliding, session windows) for aggregation — e.g. "count events per device per 5-minute window."
- **Exactly-once in Kafka Streams**: configurable via `processing.guarantee=exactly_once_v2` — under the hood, this uses the same transactional/idempotent-producer machinery this project implemented by hand (§9), automatically, for the library's internal consume-process-produce loops.

ksqlDB is a SQL-like abstraction over these same stream-processing concepts — lets you express filters, joins, and windowed aggregations declaratively in SQL rather than writing Java/Scala code, running as its own server/cluster on top of Kafka.

Q&A

Q: What's the conceptual difference between a KStream and a KTable? A: A KStream represents a stream of independent, immutable events — every record is a distinct fact. A KTable

represents the latest value per key as an evolving, queryable table — a new record for an existing key is interpreted as an update to that key's current value, not a new independent fact.

Q: Why does Kafka Streams use RocksDB-backed local state stores instead of keeping aggregation state purely in memory? A: RocksDB provides efficient on-disk storage for state that may be larger than available memory, and critically, each local state store is backed by a Kafka changelog topic — if the Streams instance crashes or is rebalanced away, its state can be fully rebuilt by replaying that changelog, giving fault tolerance without needing external distributed storage.

Q: What's ksqlDB's relationship to Kafka Streams? A: ksqlDB is a SQL abstraction layer over the same underlying stream-processing concepts (filters, joins, windowed aggregations) that Kafka Streams provides as a Java/Scala library — it runs as its own server, letting you express stream processing declaratively rather than writing and deploying custom application code.

Q: How would you compute "average device event rate per 5-minute window" using these tools, at a conceptual level? A: Group the event stream (KStream) by device ID, apply a tumbling 5-minute time window, and aggregate (count) within each window — either as Kafka Streams Java code using `.groupByKey().windowedBy(TimeWindows.of(...)).count()` -style APIs, or an equivalent ksqlDB `CREATE TABLE ... WINDOW TUMBLING (SIZE 5 MINUTES) ... GROUP BY device_id` query.

Q: If this project rewrote `DeviceEventTransactionalProcessor` using Kafka Streams instead of the hand-rolled `KafkaTransactionalProducer`, what would change? A: The consume-transform-produce-and-commit-offset loop, the retry handling around it, and the exactly-once guarantee would all become framework-managed (via `processing.guarantee=exactly_once_v2`) instead of hand-implemented — trading the fine-grained control we built (custom retry policy, custom DLT integration, explicit transaction management) for a higher-level, more declarative API that handles the same underlying mechanics automatically. It's a reasonable "how would you do this differently" follow-up question, and the honest answer is a real tradeoff, not a strictly better option.

21. Kafka Connect & CDC (Conceptual)

Not implemented — but directly relevant, since we hand-built the thing Kafka Connect + Debezium exists to replace.

Theory

Kafka Connect is a framework for moving data between Kafka and external systems via pre-built (or custom) **connectors**, without writing bespoke producer/consumer application code. **Source**

connectors pull data *into* Kafka (e.g. from a database); **sink connectors** push data *out* of Kafka (e.g. into a data warehouse, search index, or another database).

Debezium is a popular set of source connectors specifically for **Change Data Capture (CDC)** — reading a database's write-ahead log / binlog directly (e.g. Postgres's logical replication slot, MySQL's binlog) and streaming every row-level insert/update/delete as a Kafka event, in near-real-time, with no polling.

Debezium's "outbox event router" is specifically the production-grade alternative to our hand-rolled `KafkaOutboxDispatcher` : instead of a background service polling `kafka_outbox_messages` on an interval, Debezium tails the Postgres WAL and emits an event the moment the outbox row is committed — no polling latency, no polling load on the database, and it works even if the application process is down (Debezium runs independently, reading the WAL Postgres already maintains).

Tradeoff vs. our hand-rolled dispatcher

	Hand-rolled outbox dispatcher (what we built)	Debezium CDC
Latency	Bounded by poll interval (1s here)	Near-instant (WAL tailing)
DB load	Periodic polling queries	None beyond WAL read (DB already writes the WAL regardless)
Operational complexity	Just application code	Requires running/operating Kafka Connect + Debezium connector, DB logical replication configuration
Coupling	None beyond a normal DB table	Coupled to DB-specific replication log format/version
Full control over retry/backoff/dead-lettering logic	Yes, custom-built (as seen in <code>KafkaOutboxDispatcher</code>)	Connector-framework-provided, less bespoke control

Q&A

Q: What problem does Debezium's outbox event router solve, and how does it relate to what's in this codebase? A: The exact same problem as `KafkaOutboxDispatcher` — reliably publishing outbox table rows to Kafka — but by tailing the database's write-ahead log instead of polling the table on an interval. It eliminates polling latency and DB polling load, and keeps working even if the application itself is down, since it reads directly from the DB's replication stream rather than going through application code.

Q: What's the main operational cost of choosing CDC over an application-level outbox dispatcher? A: You now depend on and must operate Kafka Connect plus the Debezium connector as additional infrastructure, and you're coupled to the specific database's logical replication mechanism (e.g. Postgres's WAL/replication slots) — a real, if usually well-supported, operational and version-compatibility dependency that a plain background-service dispatcher doesn't have.

Q: What's the difference between a source connector and a sink connector? A: A source connector moves data *into* Kafka from an external system (e.g. Debezium reading DB changes into Kafka). A sink connector moves data *out of* Kafka into an external system (e.g. writing consumed events into Elasticsearch or a data warehouse).

Q: How does Debezium keep working even if the application that owns the database is down? A: It reads directly from the database's own replication mechanism (e.g. Postgres's logical replication slot) as an independent process/connector, not by calling into the application at all. As long as the database itself is up and its WAL is being written, Debezium can capture changes regardless of whether the application process happens to be running.

22. Security (Conceptual)

Not implemented — this project runs entirely unauthenticated/unencrypted (`PLAINTEXT` listeners) for local development. Know the vocabulary even without hands-on practice.

Theory

- **Encryption in transit:** `SSL / TLS` listeners encrypt broker-client and broker-broker traffic. `PLAINTEXT` (what we use) sends everything unencrypted — fine for local dev, never for production over an untrusted network.
- **Authentication:** **SASL** (Simple Authentication and Security Layer) mechanisms — `SASL/PLAIN` (username/password, should be paired with TLS), `SASL/SCRAM` (salted challenge-response, safer than PLAIN even without TLS), `SASL/GSSAPI` (Kerberos, common in enterprise/on-prem), `SASL/OAUTHBEARER` (OAuth2 token-based, common in cloud-managed Kafka). Client certificates via `mTLS` are another authentication path, layered on top of TLS.
- **Authorization:** **ACLs** (Access Control Lists) — once a client is authenticated as a specific principal, ACLs govern *what* that principal can do (produce to topic X, consume from group Y, create topics, etc.), typically managed via `kafka-acls` or a broader IAM integration on managed platforms.

- **Combined listener naming** you'll see in real configs: `SASL_SSL` (both authentication and encryption on the same listener) is the common production combination.

Q&A

Q: What's the difference between SASL/PLAIN and SASL/SCRAM, and why would SCRAM be preferred? A: PLAIN sends the username/password essentially as-is (base64, not encrypted on its own) — safe only when layered under TLS, since otherwise credentials are exposed on the wire. SCRAM uses a salted, challenge-response mechanism that never sends the actual password over the wire at all, making it meaningfully safer even in scenarios without TLS (though pairing with TLS is still standard practice).

Q: What's the difference between authentication and authorization in a Kafka security context? A: Authentication (SASL/mTLS) establishes *who* the client is — a verified principal/identity. Authorization (ACLs) then governs *what that identity is allowed to do* — which topics it can produce/consume, which consumer groups it can join, whether it can create/delete topics, etc. You need authentication first for authorization to mean anything (ACLs on an unauthenticated/anonymous connection can't distinguish callers).

Q: Why would a listener be configured as `SASL_SSL` rather than just `SASL_PLAINTEXT`? A: `SASL_PLAINTEXT` authenticates the client but still sends all traffic (including the authenticated session's data) unencrypted over the wire — vulnerable to eavesdropping/tampering in transit. `SASL_SSL` layers authentication *and* encryption together, which is the standard expectation for any production deployment over a network you don't fully trust.

Q: What's a practical reason to prefer SASL/OAUTHBEARER in a cloud-managed Kafka environment? A: It integrates with the organization's existing identity provider / OAuth2 infrastructure (short-lived tokens, centralized revocation, no long-lived static credentials to rotate manually) rather than managing separate Kafka-specific usernames/passwords or certificates — a natural fit when the rest of the cloud environment already uses OAuth2/OIDC for service-to-service auth.

23. Monitoring & Operations

Not implemented — this project didn't wire up metrics/alerting (Prometheus, Grafana), though we did use Conduktor and briefly AKHQ as Kafka management/browsing UIs during environment setup. Know the vocabulary and the reasoning conceptually.

Theory

- **Consumer lag:** the single most important operational health metric — `log-end-offset` (the partition's latest available offset) - `current-offset` (the consumer group's last committed offset) . High or growing lag means the consumer can't keep up with the produce rate; it's the primary signal for "is this consumer group healthy."
- **Key broker metrics** (exposed via JMX, commonly scraped with a Prometheus JMX exporter):
 - `UnderReplicatedPartitions` — partitions where the ISR count is below the replication factor. A leading indicator of reduced durability margin, worth alerting on even before it causes an actual outage.
 - `ActiveControllerCount` — should be exactly 1 across the whole cluster at all times; anything else indicates a controller election problem.
 - `RequestHandlerAvgIdlePercent` — broker request-handling thread saturation; low idle percent means the broker is becoming a bottleneck.
 - `BytesInPerSec` / `BytesOutPerSec` — raw throughput.
- **Key consumer-side metrics:** `records-lag-max` , `fetch-rate` , `commit-rate` , and rebalance frequency/duration (frequent or long rebalances are themselves a health signal, not just a mechanism).
- **Key producer-side metrics:** `record-error-rate` , `request-latency-avg` , `batch-size-avg` , `compression-rate` .
- **Alerting patterns:** alert on consumer lag *sustained growth over a window*, not instantaneous spikes (which are normal under bursty traffic); alert on `UnderReplicatedPartitions > 0` sustained past a grace period; alert on DLT topic message rate, since that's a direct signal of processing failures reaching the "give up" path.
- **Tooling landscape:** Prometheus + Grafana (JMX exporter for brokers, client-library metrics for producers/consumers) is the common open-source stack; Confluent Control Center is the commercial equivalent; Konduktor and AKHQ (both touched during this project's environment setup) are lighter-weight management/browsing UIs — good for ad-hoc topic/message inspection, not a substitute for metric-based alerting.

Q&A

Q: What's the single most important metric for consumer health, and how is it computed?

A: Consumer lag — the difference between the partition's log-end-offset (the latest available offset) and the consumer group's current committed offset. It directly measures "how far behind is this consumer," and sustained growth means the consumer can't keep up with the produce rate.

Q: Why would you alert on sustained lag growth rather than an instantaneous lag value?

A: Lag naturally fluctuates with normal traffic bursts and brief processing pauses — an instantaneous

threshold alert would be noisy and cause alert fatigue. Sustained growth over a window (e.g. lag increasing steadily for 10+ minutes) is a much stronger signal that the consumer genuinely can't keep pace, rather than just absorbing a temporary burst.

Q: What does a nonzero `UnderReplicatedPartitions` broker metric indicate, and why does it matter? A: It means at least one partition has fewer in-sync replicas than its configured replication factor — some follower(s) have fallen behind the leader. It's a leading indicator of reduced durability margin (closer to violating `min.insync.replicas` if another replica also falls behind or fails) and should be alerted on even before it causes an actual outage.

Q: What tools did we actually touch in this project relevant to Kafka observability? A: Conduktor (a full Kafka management UI, running as part of the local environment) and briefly AKHQ, during environment setup — both give topic/message browsing and basic cluster visibility, though we didn't wire up metric-based alerting (Prometheus/Grafana) in this project; that's a real gap worth naming honestly if asked what's missing operationally.

Q: Besides consumer lag, what's a leading indicator you'd want to alert on for the *producer* side of this specific system? A: The outbox table's own retry/failure state — e.g. a growing count of rows stuck in `Failed` status with rising `retry_count`, or any row reaching `DeadLettered` status. That's a direct signal of persistent publish failures (broker connectivity, schema registry incompatibility, etc.) surfaced from application-owned state, independent of and complementary to Kafka-level metrics.

Q: If you saw `ActiveControllerCount` reporting a value other than 1, what would that suggest? A: A controller election problem — either no broker currently believes it's the active controller (0, likely mid-election or a cluster-wide issue), or a split-brain-like situation where more than one broker believes it holds the role (should be architecturally prevented by the consensus protocol, but a nonzero-and-not-1 reading is a serious signal worth immediate investigation either way).

24. Testing Kafka Applications

Not implemented — this project has no automated test suite; verification throughout this session was done by hand against a live Docker Compose stack. Worth knowing the standard testing approaches, and being honest that this project itself doesn't demonstrate them.

Theory

- **Unit testing business logic:** keep Kafka-specific concerns (serialization, offset management, transaction plumbing) out of core business logic so it's testable without any Kafka dependency at

all. E.g. `DeviceEventHandler` 's actual processing logic could be unit tested by constructing a `DeviceEvent` directly and asserting on the resulting side effects, without touching Kafka at all.

- **Integration testing against a real broker: Testcontainers** (spins up a real, ephemeral Kafka broker in Docker for the duration of a test run, then tears it down) is the standard modern approach — meaningfully more faithful than mocking the Kafka client, since Kafka's client behavior (partitioning, consumer group coordination, transactional semantics) is genuinely hard to mock convincingly and easy to get subtly wrong in a hand-rolled mock.
- **Contract testing with Schema Registry**: verifying a producer's schema change doesn't break registered consumers *before* deploying — done by running the actual compatibility-check dry-run endpoint (`POST /compatibility/subjects/{subject}/versions/latest`) as a CI step against a real or test Schema Registry instance. This is exactly the technique used manually, by hand, in this project's own compatibility testing (§15) — the natural next step is automating it.
- **Testing idempotency**: a meaningful idempotent-consumer test explicitly delivers the same message twice and asserts the end state is identical to delivering it once — not just "does the code avoid throwing an exception on the second delivery."
- **Testing DLT behavior**: assert that a message engineered to fail actually lands on the DLT topic with the expected diagnostic headers, *and* that the main topic's consumer offset still advances past it (i.e. the partition isn't stuck).

Q&A

Q: Why is mocking the Kafka client generally a weaker testing strategy than using

Testcontainers? A: Kafka's client behavior (consumer group coordination, partition assignment, transactional semantics, retry/backoff behavior) is complex and stateful — a hand-rolled mock is very likely to diverge from real broker behavior in ways that hide real bugs. In fact, several of the real bugs covered in this guide's "real bugs" section (§26) — like the transaction-poisoning issue, which only manifests through the *actual* stateful behavior of a real transactional producer client — would likely never be caught by a mock, since a mock would have to deliberately reimplement that exact subtle state machine to expose the bug. Testcontainers runs an actual Kafka broker for the test, giving much higher fidelity at the cost of slower test execution.

Q: How would you test that this project's schema changes don't break existing consumers before deploying?

A: Run the same compatibility dry-run check used manually in this project (`POST /compatibility/subjects/{subject}/versions/latest`) as an automated CI step against the target Schema Registry, failing the build if the proposed schema is incompatible with the currently-registered one under the subject's configured compatibility mode.

Q: What's the key assertion in a proper idempotency test, beyond "the code doesn't

throw"? A: That delivering the same message twice produces the same end state as delivering it once —

e.g. assert a downstream side effect (a DB row, a counter) reflects exactly one application of the effect, not "did processing complete without an exception" (which a non-idempotent handler could also satisfy on both deliveries, just with the wrong cumulative effect, like a counter incremented twice instead of once).

Q: What should a DLT behavior test actually verify? A: That a message engineered to fail (a bad payload, a simulated exception) actually appears on the dead-letter topic with the expected diagnostic headers (original topic/partition/offset, exception type/message), AND that the consumer's offset on the main topic advances past the poisoned message rather than getting stuck redelivering it forever.

Q: This project doesn't have an automated test suite — how was correctness actually verified throughout this session? A: By hand, against a live, real Docker Compose stack — running the actual producer/consumer/Kafka/Postgres/Schema Registry containers, inspecting real logs, querying the real outbox table, and using the Schema Registry's actual REST API for compatibility checks. This gave high-fidelity, ground-truth verification (several bugs were only found this way, not from reading code), but it's manual and not repeatable/automatable the way a Testcontainers-based test suite would be — an honest gap worth naming if asked "how would you make this more production-ready."

25. Docker/KRaft Setup — What We Actually Hit

This is worth including because it's a genuinely common real-world gotcha, and we hit it more than once while building this environment.

The `advertised.listeners` trap

A Kafka broker's `advertised.listeners` config tells clients *where to reconnect* after the initial bootstrap connection — this is what gets embedded in the metadata response every client uses for all follow-up requests, not just the first one. If a broker is only configured with a single listener advertised as, say, `localhost:9092`, that works fine for a client running on the *host machine* (where "localhost" correctly means "this machine"). But a client running *inside another Docker container* on the same Docker network, connecting via the container's Docker-DNS hostname, will successfully complete the *initial* bootstrap connection — then receive metadata telling it to reconnect to `localhost:9092`, which inside its own container means *itself*, not the broker. The connection then mysteriously fails on every subsequent request, even though the first connection appeared to work.

Fix: configure *two* listeners — one advertised for the Docker-internal network (e.g.

`PLAINTEXT://kafka:29092`), one advertised for the host machine (`PLAINTEXT_HOST://localhost:9092`):

```
KAFKA_LISTENERS: 'PLAINTEXT://kafka:29092,CONTROLLER://kafka:29093,PLAINTEXT_HOST://0.0.0.0:9092'
KAFKA_ADVERTISED_LISTENERS: 'PLAINTEXT://kafka:29092,PLAINTEXT_HOST://localhost:9092'
KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: 'CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT'
KAFKA_INTER_BROKER_LISTENER_NAME: 'PLAINTEXT'
```

Other containers on the same Docker network use `kafka:29092` ; anything on the host machine (including `dotnet run` outside Docker) uses `localhost:9092` .

Q&A

Q: A container can make an initial connection to a Kafka broker in another container, but every subsequent request fails. What's the likely cause? A: `advertised.listeners`

misconfiguration — the broker is telling clients to reconnect to an address (commonly `localhost`) that only resolves correctly from the broker's own machine/container, not from the calling container's network namespace. The fix is a dual-listener setup: one address advertised for same-network container traffic (the Docker-internal hostname), one for host-machine traffic.

Q: Why does the *first* connection succeed but later ones fail in this scenario? A: The first connection is a raw bootstrap connection to whatever address was configured client-side (e.g. the Docker Compose service name, which Docker's internal DNS resolves correctly). Every request *after* that uses the broker's own self-reported `advertised.listeners` metadata to decide where to connect — and that's where the misconfigured, context-dependent address (`localhost`) breaks for anyone not on the broker's own host/network namespace.

Q: Why can't you just always use the Docker-internal hostname (e.g. `kafka:29092`) everywhere, including from the host machine, to sidestep this entirely? A: Docker-internal hostnames are only resolvable via Docker's internal DNS, which is scoped to containers on that Docker network — a process running natively on the host machine (like `dotnet run` outside a container) has no way to resolve `kafka` as a hostname at all. You genuinely need the dual-listener setup because host-machine clients and container clients resolve names through fundamentally different DNS scopes.

26. Real Bugs We Debugged — Interview Gold

"Tell me about a challenging bug you fixed" is one of the most common interview questions, and generic answers are weak. These are real, specific, and each demonstrates a different debugging skill. Know the story, the root cause, and the fix for each.

Bug 1: NuGet package downgrade conflict

Symptom: NU1605 error on restore after adding `Confluent.SchemaRegistry`. **Root cause:** The new package's transitive dependency (`Microsoft.Extensions.Caching.Memory`) required `Microsoft.Extensions.Logging.Abstractions >= 8.0.2`, but the project explicitly pinned it to `8.0.0` — a direct downgrade conflict. Complicated by the fact that `8.0.2` doesn't actually exist as a published version for that specific package, so the resolver kept jumping to `9.0.0`, cascading further version mismatches until every `Microsoft.Extensions.*` reference was aligned to `9.0.0`. **Skill demonstrated:** Reading NuGet dependency-resolution error chains precisely, understanding transitive version constraints, not just blindly bumping numbers until it compiles.

Bug 2: Wrong assumption about async/sync serializer support

Symptom: Compile error `CS1503: cannot convert from IAsyncDeserializer<TValue> to IDeserializer<TValue>` after changing the consumer to accept an async deserializer (assuming, incorrectly, that `ConsumerBuilder` supported async deserializers symmetrically with `ProducerBuilder`'s async serializer support). **Root cause:** `ProducerBuilder<TKey, TValue>` genuinely supports both sync and async value serializers. `ConsumerBuilder<TKey, TValue>` only supports the synchronous `IDeserializer<T>` — there's no async overload, because deserialization happens inline during `Consume()`, not as a separately awaitable operation. **Fix:** Kept the consumer-side deserializer synchronous, and bridged the inherently-async schema-registry `JsonDeserializer<T>` into that shape using `Confluent.Kafka.SyncOverAsync`'s `.AsSyncOverAsync()` extension. **Skill demonstrated:** Verifying an assumption against the actual API surface (via compiler error, not guesswork) rather than doubling down on an incorrect mental model.

Bug 3: Transaction-poisoning bug (the subtlest one)

Symptom: After one non-retriable Kafka exception mid-transaction, *every subsequent* transactional produce failed with `Operation not valid in state InTransaction` — a single failure permanently broke the transactional producer for the rest of the process's lifetime. **Root cause:** `KafkaTransactionalProducer.AbortIfRequired` had three branches: fatal errors (log, don't abort — arguably correct since the client is dead anyway), `KafkaTxnRequiresAbortException` (correctly aborts), and a catch-all default branch that **only logged the error and returned, without calling `AbortTransaction()`**. Since `BeginTransaction()` had already succeeded, the underlying producer was left in the "in transaction" state indefinitely — no code path ever called `Commit` or `Abort` to close it out. **Fix:** Made the catch-all branch unconditionally call `AbortTransaction()` for any non-fatal exception, since `BeginTransaction()` having succeeded means *something* must close the transaction before the producer is usable again. **Skill demonstrated:** Diagnosing a stateful, session-scoped failure mode (not visible from a single stack trace — required understanding that the *producer instance* itself was permanently corrupted, not just one failed call) and fixing the actual state-machine invariant, not just the symptom.

Bug 4: Schema-registry-based serializer incompatible with sync `Produce()`

Symptom: `InvalidOperationException: Produce called with an IAsyncSerializer value serializer configured but an ISerializer is required` — thrown deep inside the Confluent client, inside an active Kafka transaction. **Root cause:** `KafkaTransactionalProducer.ProduceOne` used the synchronous, fire-and-forget `_producer.Produce(...)` (the idiomatic pattern for transactional multi-message produces, since you typically don't want to await each one individually) — but the value serializer was now the async schema-registry `JsonSerializer<T>`, and `Produce()` (unlike `ProduceAsync()`) categorically cannot use an async serializer. **Fix:** Changed to `await _producer.ProduceAsync(...)`, and changed `ExecuteInTransactionAsync`'s `produceMessages` parameter from `Action` to `Func<Task>`, awaiting it inline within the transaction. A real tradeoff accepted: sequential awaited produces instead of fire-and-forget batching, in exchange for correctness with the async serializer. **Skill demonstrated:** Understanding a real constraint at the intersection of two separate features (Kafka transactions' idiomatic produce pattern vs. Schema Registry's inherently async serialization), and choosing a correct, if less optimal, fix over a workaround.

Bug 5: JSON casing mismatch between producer and outbox dispatcher

Symptom: `JsonException: JSON deserialization for type 'DeviceEvent' was missing required properties` — thrown by the outbox dispatcher trying to read back its own outbox table rows. **Root cause:** `KafkaOutboxMessageFactory` serialized the outbox payload using `JsonSerializerDefaults.Web` (camelCase property names). `KafkaOutboxDispatcher.PublishMessageAsync` deserialized it with a bare, default `JsonSerializer.Deserialize<DeviceEvent>(json)` call — no options, meaning case-sensitive PascalCase matching. camelCase JSON against a case-sensitive PascalCase-expecting deserializer, on a record with `required` members, failed every single field. **Fix:** Deserialize using the same `JsonSerializerOptions(JsonSerializerDefaults.Web)` on both sides. **Skill demonstrated:** Recognizing that "the same logical operation" (serialize/deserialize a DTO) performed in two different places in a codebase needs *consistent* configuration, not just individually-reasonable-looking code at each site — a classic "it compiles, it looks fine in isolation, but the two halves don't agree" bug.

Bug 6: The "consumer validates against the registry's latest schema, not the CLR type" gotcha

Symptom: After adding an optional `Severity` field to `DeviceEvent` and rebuilding only the consumer, replaying old (pre- `Severity`) messages failed with `Schema validation failed for properties: [#/Severity]` — even though the schema itself, tested in isolation with `NJsonSchema`'s own validator, correctly accepted the old-shaped payload with zero errors. **Root cause:** Confirmed by reading Confluent's actual SDK source: `JsonDeserializer<T>.DeserializeAsync` re-serializes the deserialized object and validates that round-tripped JSON against the subject's *latest registered schema in the registry* — not the writer's original schema, and not the local CLR type in isolation. Since only the

consumer had been rebuilt (the producer hadn't registered a schema version including `Severity` yet), the registry's "latest" schema for that subject was still the old, `Severity`-less, closed-content-model version — and the consumer's own re-serialized object (now including an explicit `"Severity": null`) got rejected as an undeclared property by that stale registered schema. **Fix:** Rebuilt and restarted the producer too, so it auto-registered a new schema version including `Severity` — only then did the consumer's post-validation step succeed, because "latest" now matched the current type. **Skill demonstrated:** Not accepting a plausible-but-wrong hypothesis (the schema itself was somehow wrong) at face value — verifying it empirically (isolated schema validation test showed zero errors), then going to primary-source SDK code to find the *actual* mechanism, rather than guessing indefinitely. This is one of the strongest "how do you debug something you don't understand" stories available from this project.

Bug 7: Test pollution blocking a legitimate schema registration

Symptom: After correctly fixing the `Severity` /content-model issue, the producer *still* failed to register the corrected schema — `SchemaRegistryException: Schema being registered is incompatible with an earlier schema`, citing `PROPERTY_ADDED_TO_OPEN_CONTENT_MODEL` and `ADDITIONAL_PROPERTIES_REMOVED`. **Root cause:** An earlier, exploratory experiment (temporarily switching the schema generator to an *open* content model to test FORWARD compatibility) had left an actual *open-model* schema version registered in the live registry as "latest." The new, correctly-closed-model schema (with `Severity`) was being compared against that stray open-model version, not against the real production baseline (`v1`), and the open→closed transition itself was flagged as incompatible on top of the field addition. **Fix:** Temporarily set the subject's compatibility mode to `NONE` to register the correct schema past the polluted version, then restored the compatibility mode to the normal default — a deliberate, understood, temporary override, not a silent workaround. **Skill demonstrated:** Distinguishing "the system is telling me something true about my current registered state" from "the system is telling me my new schema is fundamentally wrong" — recognizing that the actual blocker was leftover state from earlier testing, not a flaw in the fix itself, and resolving it explicitly rather than fighting the compatibility checker or disabling it permanently.

Q&A

Q: Walk me through the transaction-poisoning bug — what was actually broken and why was it subtle? **A:** A non-fatal exception during a Kafka transaction (after `BeginTransaction()` had already succeeded) hit a code path that logged the error but never called `AbortTransaction()`. Since transactions are a stateful session on the underlying producer client (not a stateless per-call operation), that left the producer instance permanently stuck believing it was mid-transaction — every future `BeginTransaction()` call on that same producer instance failed immediately. It was subtle because the *symptom* (every subsequent message failing) looked unrelated to the *original* failure, and nothing in a single stack trace pointed at "the producer object itself is now permanently broken" — that required understanding the transaction state machine, not just the immediate exception.

Q: Describe a bug where your initial hypothesis was wrong, and how you found the real cause. A: The `Severity` field / schema validation bug. My first hypothesis was that the generated JSON schema itself incorrectly required or mishandled the new nullable field. I tested that in isolation — generating the schema and validating an old-shaped payload against it directly — and got zero validation errors, disproving the hypothesis. Rather than keep guessing, I went to the actual SDK source for `JsonDeserializer<T>` and found it re-validates against the schema registry's latest registered version after re-serializing the deserialized object — a mechanism with no obvious hint from the error message alone. The real fix (register a new producer-side schema version first) followed directly once the actual mechanism was understood.

Q: Give an example of a bug caused by two pieces of "correct-looking" code disagreeing with each other. A: The JSON casing mismatch between `KafkaOutboxMessageFactory` (serializing with camelCase `JsonSerializerDefaults.Web`) and `KafkaOutboxDispatcher` (deserializing with default, case-sensitive PascalCase options). Neither piece of code was wrong *in isolation* — each was a perfectly ordinary `JsonSerializer` call — but they didn't share configuration, so the second couldn't correctly read what the first wrote. The fix was ensuring both sides used identical `JsonSerializerOptions`.

Q: Describe a case where the "error" you were seeing was actually accurate, but about the wrong thing. A: The test-pollution schema registration failure. The compatibility checker's rejection was completely accurate — the new schema genuinely was incompatible with what was currently registered as "latest." The mistake would have been assuming that meant the *fix itself* was wrong. The real issue was that "latest" was itself polluted by an earlier, unrelated experiment. Recognizing that distinction — "this error is correct, but about stale state, not about my actual change" — is what let me resolve it properly (clear the stale state deliberately) instead of either fighting the checker or disabling compatibility checking permanently as a workaround.

27. Rapid-Fire Q&A Catalog

A final pass — quick-recall format for last-minute revision. Cover the answer and see if you can produce it in one or two sentences.

1. **What guarantees does Kafka give about message ordering?** Per-partition only, never across partitions.
2. **What determines which partition a keyed message goes to?** A hash of the key (by default), modulo the current partition count.
3. **What's the difference between `acks=1` and `acks=all`?** `acks=1` waits only for the partition leader; `acks=all` waits for all current in-sync replicas.

4. **What does `EnableIdempotence=true` prevent?** Broker-level duplicate appends from producer-session retries (via PID + per-partition sequence numbers).
5. **What's the difference between the idempotent producer and Kafka transactions?** The idempotent producer dedups retries within a single producer session on a single partition; transactions atomically group multiple produces (potentially across partitions/topics) plus a consumer offset commit into one all-or-nothing unit.
6. **Why must `EnableAutoCommit` be `false` for reliable at-least-once processing?** Auto-commit advances offsets on a timer, independent of whether processing actually succeeded — risking silent message loss on a crash between auto-commit and completed processing.
7. **What's the dual-write problem, and what pattern solves it?** Writing to a database and independently producing to Kafka aren't atomic — one can succeed while the other fails. The outbox pattern solves it by writing the "event to publish" as a row in the same DB transaction as the business write.
8. **What makes the outbox dispatcher safe to run as multiple instances?** `SELECT ... FOR UPDATE SKIP LOCKED` — each instance claims a disjoint set of unlocked rows instead of blocking on others' locks.
9. **What's the difference between at-least-once and exactly-once?** At-least-once can redeliver/duplicate but never silently loses a message; exactly-once ensures the observable *effect* of processing happens once, via transactions or idempotent processing.
10. **Why is a strict retry allow-list (only specific exception types) better than "retry everything"?** Permanent failures (bad data, bugs) will fail identically on every retry — retrying them wastes time and can block the whole partition, since Kafka can't skip past an uncommitted offset.
11. **What's the purpose of a dead letter topic?** Isolate a poison message so the consumer can keep making progress on subsequent messages, while preserving the failed message (with failure context) for later inspection or replay.
12. **Why must you explicitly commit the offset after publishing to a DLT?** Otherwise Kafka redelivers the same poison message forever, since the last committed offset is still before it — an infinite loop that blocks the partition.
13. **Why is idempotent consumption necessary even with a reliable outbox and manual offset commits?** Both mechanisms independently guarantee at-least-once, not exactly-once — duplicates are an expected, legitimate outcome, and only idempotent processing converts that into an exactly-once *effect*.
14. **What's wrong with an in-memory idempotency/dedup store for production?** It's per-process state, wiped on every restart — a message redelivered after a restart won't be recognized as a duplicate.

15. **What does a Schema Registry compatibility mode actually enforce, and when?** It's checked when a producer attempts to register a new schema version — an incompatible change is rejected at publish time (a 409), not discovered later by a crashing consumer.
16. **BACKWARD compatibility, one sentence.** New-schema consumers can still read data written by the previous schema.
17. **FORWARD compatibility, one sentence.** Old-schema consumers can still read data written by the new schema.
18. **Does removing a field ever pass any compatibility mode?** No — deletion breaks BACKWARD, FORWARD, and FULL; the safe practice is deprecating (keep declared/nullable) rather than removing.
19. **What's the practical effect of a closed content model (`additionalProperties: false`) on compatibility?** It favors BACKWARD for additive optional-field changes but breaks FORWARD/FULL for the same change, since the old closed schema rejects the new undeclared field.
20. **Why can adding a field break a consumer even for data produced before the field existed?** Because the deserializer's post-validation step checks against the registry's *current latest* schema, not the writer's original schema — if no producer has registered a version including the new field yet, that validation fails regardless of what the actual message bytes contain.
21. **What's the wire format for a Confluent schema-registry-encoded message?** 1 magic byte + 4-byte big-endian schema ID + the payload.
22. **What does `session.timeout.ms` detect that `max.poll.interval.ms` doesn't, and vice versa?** Session timeout detects a consumer that's stopped sending heartbeats (likely dead/disconnected). Max poll interval detects a consumer that's still heartbeating (alive) but stuck and not calling `poll()` — a different failure mode.
23. **What's the practical difference between eager and cooperative-sticky partition assignment?** Eager revokes all partitions from all consumers on any rebalance trigger (stop-the-world); cooperative-sticky only moves the specific partitions that need to change, leaving unaffected consumers uninterrupted.
24. **What is `min.insync.replicas` for?** It sets the minimum ISR size required for an `acks=all` produce to succeed — prevents silently downgrading durability when ISR shrinks (e.g. broker failures) by making produces fail loudly instead.
25. **Why must a new partition leader be elected only from ISR members?** Non-ISR replicas are lagging and missing recent, acknowledged writes; electing one as leader would silently lose data that clients were told was durably written.
26. **What replaced ZooKeeper in modern Kafka, and how does it work?** KRaft — a subset of the Kafka brokers themselves act as controllers, managing cluster metadata via the Raft consensus protocol, removing the separate ZooKeeper dependency.

27. **What does log compaction retain that time/size-based retention doesn't?** The latest message per key, indefinitely — used for "current state" topics rather than "event history for N days" topics.
28. **What problem does Kafka Connect + Debezium's CDC approach solve better than a polling outbox dispatcher?** Near-instant latency (WAL tailing vs. poll-interval-bound) and zero DB polling load, at the cost of additional infrastructure (Kafka Connect) and coupling to the DB's specific replication log format.
29. **SASL/PLAIN vs. SASL/SCRAM — which leaks less even without TLS, and why?** SCRAM — it's a salted challenge-response mechanism that never sends the actual password over the wire, unlike PLAIN.
30. **What's the difference between authentication and authorization in Kafka security?** Authentication (SASL/mTLS) establishes *who* the client is; authorization (ACLs) governs what that authenticated identity is permitted to do.
31. **Why did our project need dual Kafka listeners in Docker Compose?** A broker's `advertised.listeners` value is what clients reconnect to for every request after the first — a single listener advertised as `localhost` works for host-machine clients but breaks for clients in other containers, which need a Docker-network-resolvable hostname instead.
32. **What's the real reason `KafkaTransactionalProducer.ProduceOne` had to switch from `Produce()` to `ProduceAsync()` ?** The async schema-registry serializer categorically cannot be used with the synchronous, fire-and-forget `Produce()` call — only `ProduceAsync()` supports async value serializers.
33. **Why did one uncaught, non-aborted transaction failure break every subsequent transactional produce on that producer instance?** Transactions are stateful on the producer client itself — once `BeginTransaction()` succeeds, *something* must call commit or abort before the client is usable again; a code path that only logged and returned left it permanently stuck "in transaction."
34. **What's the tradeoff of raising a producer's `linger.ms` ?** Better throughput and compression (bigger batches), at the cost of added per-message latency (up to `linger.ms` in the worst case) while the producer waits to batch.
35. **Why is partition count effectively a one-way decision?** Kafka supports growing partition count but not shrinking it on an existing topic — reducing it would require redistributing or discarding data with no clean semantics, so over-provisioning is a real, hard-to-reverse cost.
36. **Why is Kafka's sequential-write, page-cache-reliant design faster than a naive random-access, self-caching alternative?** Sequential I/O avoids costly seeks; relying on the OS page cache avoids double-buffering the same data in both the JVM heap and the OS cache, and enables zero-copy transfer of already-cached data straight to the network socket.

37. **What is zero-copy, concretely, and why does it matter for Kafka's throughput?** The `sendfile()` syscall transfers bytes from a file/page cache directly to a network socket in kernel space, skipping the copy into and back out of the application's user-space memory — meaningfully reducing CPU and memory-bandwidth overhead when serving fetch requests from cached log segments.
38. **What's the core consumption-model difference between Kafka and a traditional push-based message queue?** Kafka consumers pull at their own pace via `poll()`, giving natural backpressure; traditional queues push messages to consumers (bounded by a prefetch limit), which requires careful tuning to avoid overwhelming a slow consumer.
39. **How does Kafka natively support multiple independent applications consuming the same full event stream, where a traditional queue typically can't without extra setup?** Messages aren't deleted on consumption — separate consumer groups each track their own offsets and can independently read the entire topic at their own pace, with no fanout/duplication configuration required.
40. **What's the purpose of a message header versus putting the same information in the payload?** Headers carry metadata about the message (tracing, routing, versioning context) that's readable without deserializing the full value, and doesn't require changing or polluting the value's own schema.
41. **How does this project implement basic distributed tracing across multiple Kafka hops?** By propagating a `correlation-id` header from the original event onto every derived event produced downstream (e.g. the transactional processor copies the inbound event's correlation ID onto the audit event it produces), letting you trace one logical business event across topics even though each hop is a technically distinct message.
42. **Why doesn't Kafka support a standard two-phase commit (2PC) across itself and an external database?** It doesn't implement an XA/2PC participant interface, and 2PC is a blocking protocol prone to leaving participants "in doubt" on a crash mid-protocol — which is exactly why patterns like the transactional outbox exist, avoiding the need for distributed-transaction coordination between two heterogeneous systems.
43. **What's the scope of what Kafka's own transactions guarantee atomicity over?** Only operations within Kafka itself (multiple produces plus a consumer offset commit) — not atomicity with an external system like a database in the same transaction.
44. **What's the claim-check pattern, and why would you use it with Kafka?** Storing a large payload in external blob storage and publishing only a lightweight reference to Kafka — used to avoid degrading replication traffic, page cache efficiency, and batching for all messages sharing a topic/broker, which large inline payloads would otherwise do.
45. **What's the conceptual difference between a KStream and a KTable in Kafka Streams?** A KStream is a stream of independent events; a KTable represents the latest value per key as an

evolving table, where a new record for a key updates its current value rather than being treated as an independent fact.

46. **How does Kafka Streams achieve fault tolerance for stateful operations like aggregations?** Local RocksDB-backed state stores are themselves backed by a Kafka changelog topic — if an instance crashes or is rebalanced away, its state can be rebuilt by replaying that changelog, without needing external distributed storage.
 47. **What's ksqlDB's relationship to Kafka Streams?** A SQL abstraction layer over the same stream-processing concepts, running as its own server, letting you express stream processing declaratively instead of writing custom Java/Scala application code.
 48. **What's a "zombie consumer," and how does it connect to why idempotent processing matters?** A consumer that appears dead to the coordinator (e.g. a GC pause exceeding session timeout) and has its partitions reassigned, but is actually still alive and finishes processing its last batch — potentially overlapping with the new partition owner's processing of the same messages. Idempotent processing turns that overlap into safe, redundant work instead of corrupted state.
 49. **What's a producer epoch, and how does it relate to the zombie consumer scenario, conceptually?** Both are fencing mechanisms for the same underlying class of problem — a "zombie" instance (producer or consumer) that appears dead but is still active and could interfere with whatever has taken over its role. Producer epochs fence zombie producers out of committing stale transactions; idempotent consumption protects against zombie consumers' overlapping reprocessing.
 50. **Why is mocking the Kafka client a weak testing strategy for something like the transaction-poisoning bug covered in this guide?** That bug only manifests through the real, stateful behavior of an actual transactional producer client across multiple calls — a hand-rolled mock would have to deliberately reimplement that exact subtle state machine to expose it, which defeats the point of testing against unknown behavior. Testcontainers, using a real broker, would actually be capable of surfacing it.
 51. **What's the single most important consumer-side metric to monitor, and how is it computed?** Consumer lag — `log-end-offset - current committed offset` for a partition — the primary signal for whether a consumer group is keeping up with the produce rate.
 52. **Why alert on sustained lag growth instead of an instantaneous lag threshold?** Lag naturally fluctuates with normal bursty traffic; sustained growth over a window is a much stronger, less noisy signal that the consumer genuinely can't keep pace.
 53. **What does a nonzero `UnderReplicatedPartitions` metric indicate?** At least one partition has fewer in-sync replicas than its configured replication factor — a leading indicator of reduced durability margin, worth alerting on before it becomes an actual outage.
-

End of guide. Revisit §26 and §27 the night before an interview — they're the highest-density recall material.